UCS3461 Foundations of Artificial Intelligence

UNINFORMED SEARCH

Semester 4, B.E. Computer Science and Engineering
15 December 2025

Sakaya Milton R
SSN College of Engineering

Depth-First Search

Exhaustive Enumeration of Paths

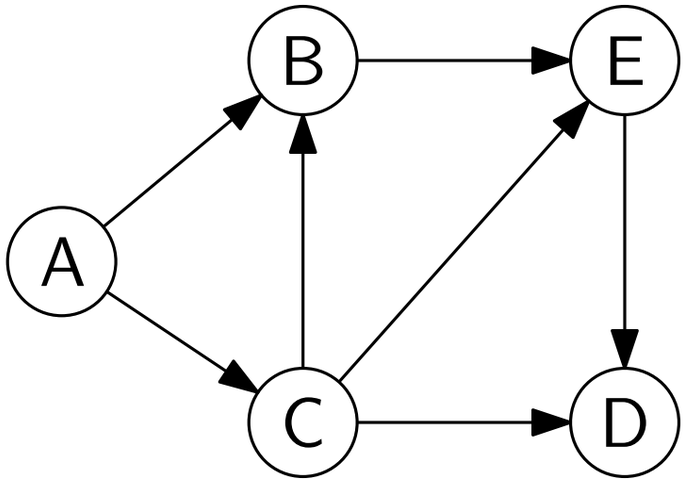
Algorithm: Exhaustive_Enumerate(G, [s..v])

Input: G = (V, E): DAG
[s..v]: path

Output: prints [s..v]

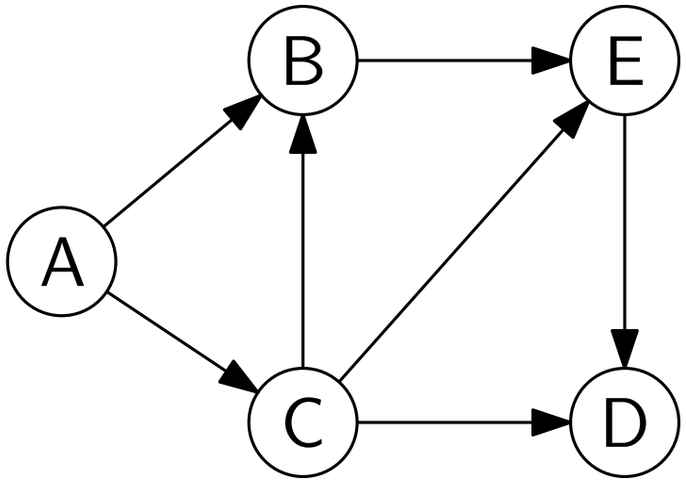
print ([s..v])
foreach $vw \in E$ **do**
    Exhaustive_Enumerate(G, [s..v,w])

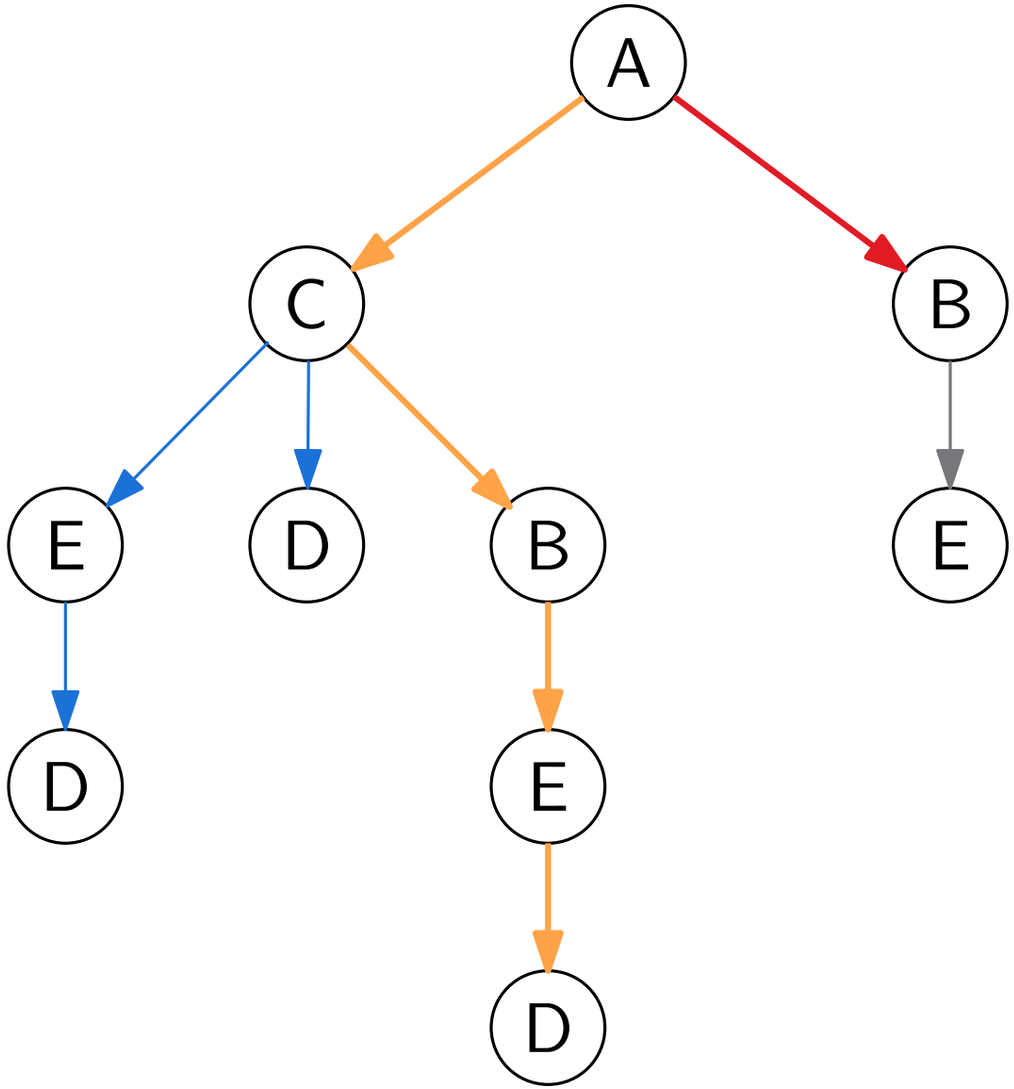
Exhaustive Enumeration of Paths



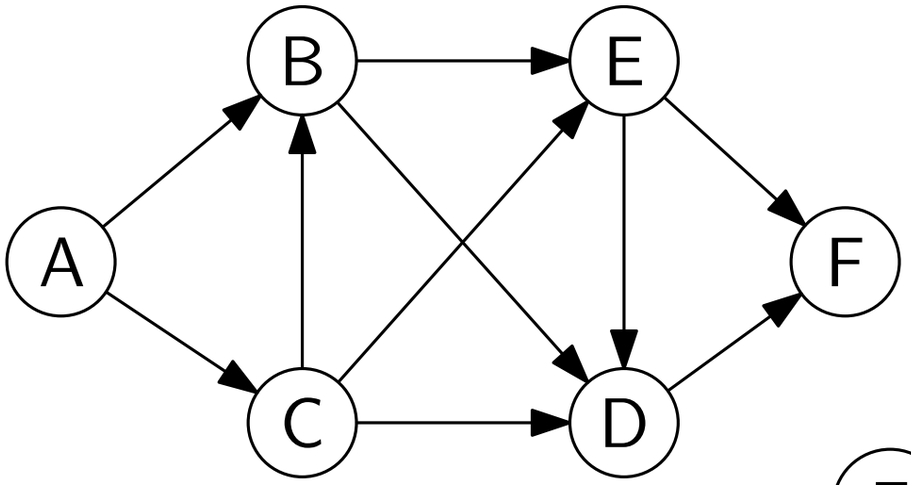
_A	AB, AC
AC	AB | ACB, ACD, ACE
ACE	AB | ACB, ACD | ACED
ACED	AB | ACB, ACD
ACD	AB | ACB
ACB	AB | ACBE
ACBE	AB | ACBED
ACBED	AB
AB	ABE
ABE	ABED
ABED	

Recursion Tree

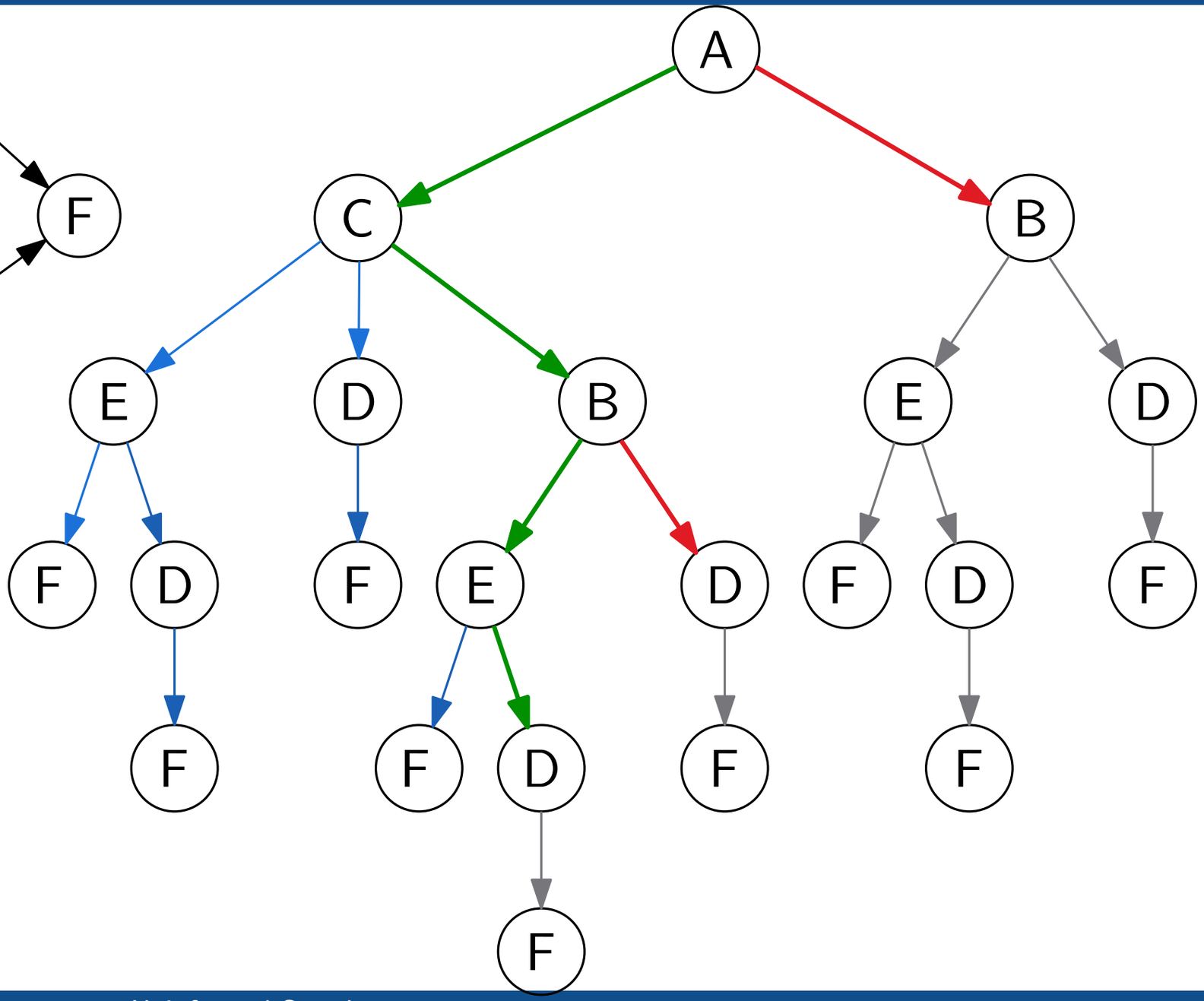




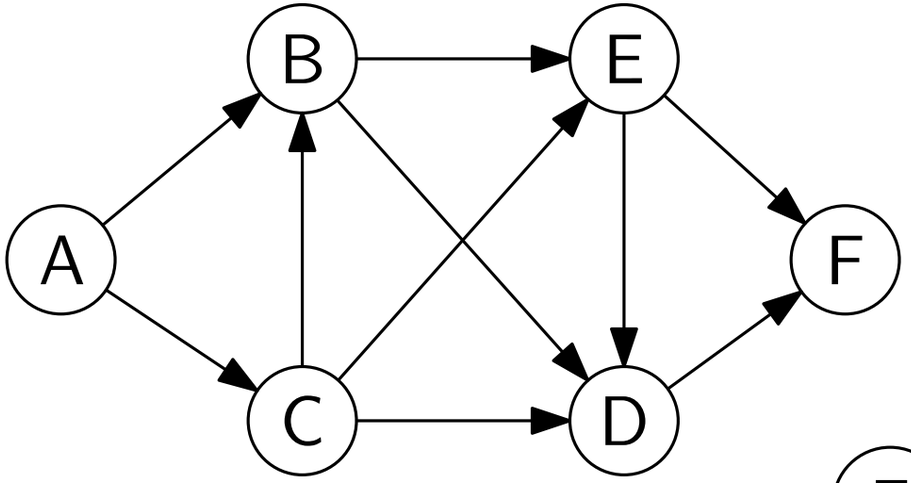
Recursion Tree for Exhaustive Enumeration of Paths



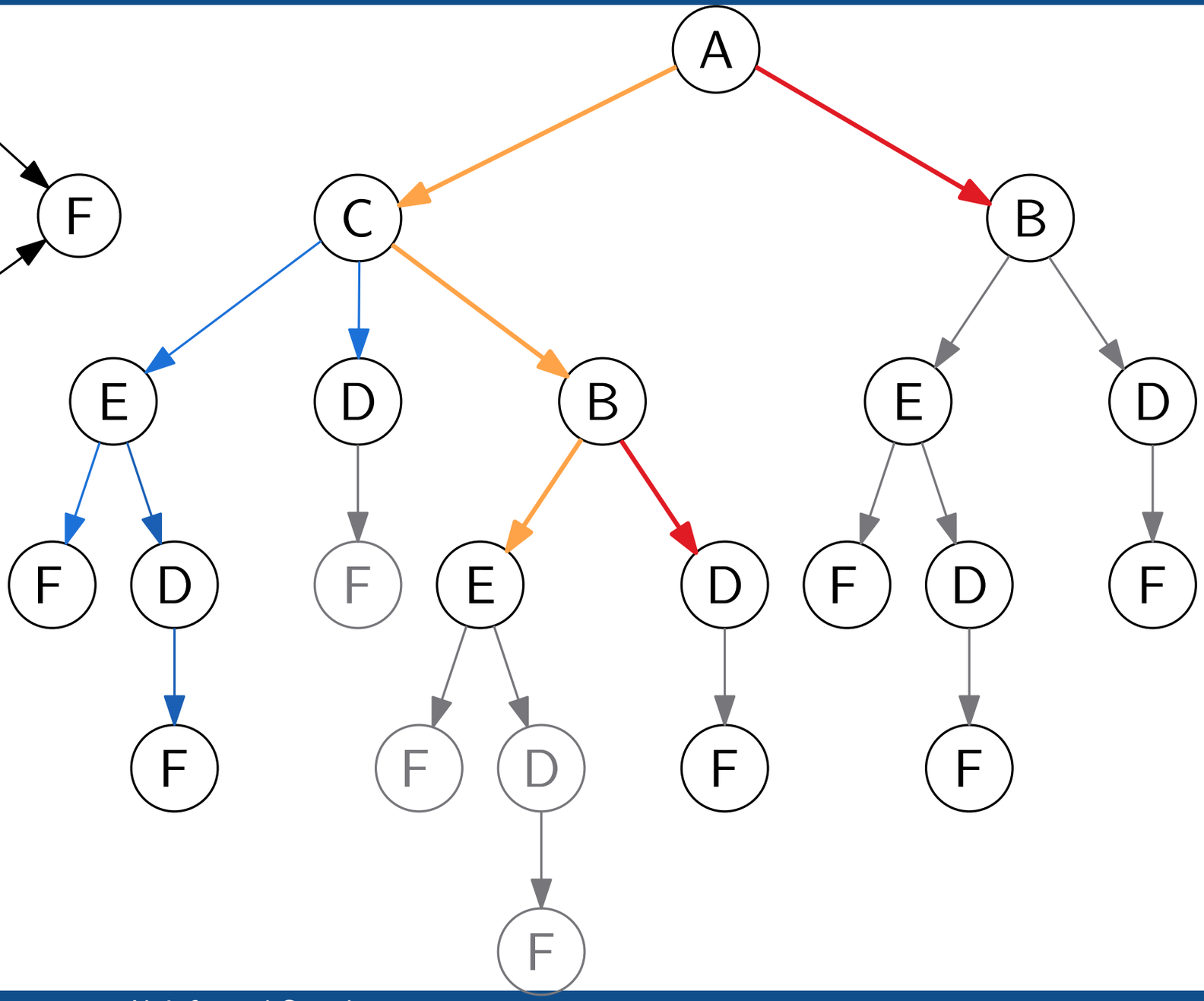
- Finished calls: called and returned
- Active calls: called but not returned
- Pending calls
- Uncalled



Recursion Tree for Explored Vertices



- Finished calls: called and returned
- Active calls: called but not returned
- Pending calls
- Uncalled



Depth-First Search

Algorithm: Depth_First_Search([s..v])

Input: [s..v]: path
Output: prints [s..v]
Global: G = (V, E): DAG
explored: set of explored vertices

if v is explored **then**
    return
add v to explored
print [s..v]
foreach vw $\in$ E **do**
    Depth_First_Search([s..v,w])

- Exhaustive enumeration of explored vertices.

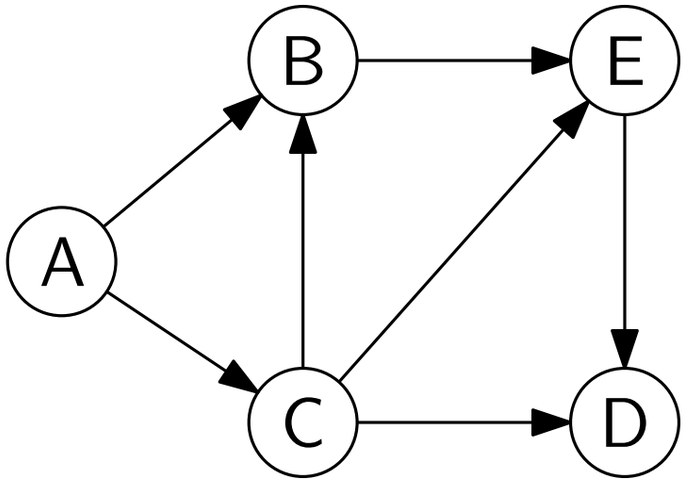
Depth-First Search

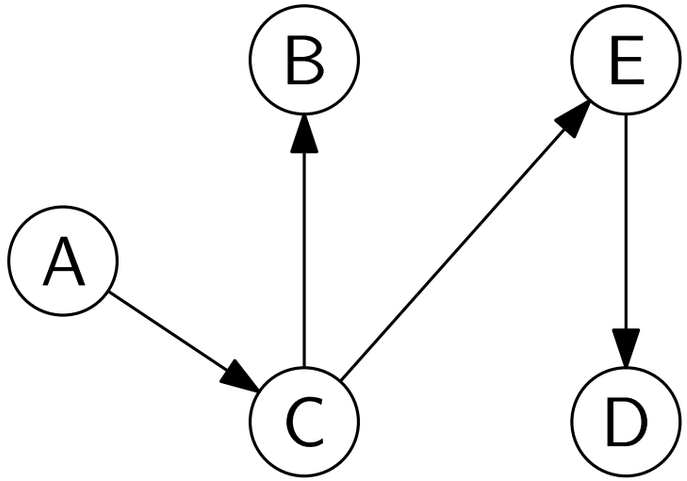
Algorithm: Depth_First_Search([s..v])

Input: [s..v]: path
Output: prints [s..v]
Global: G = (V, E): DAG
explored: set of explored vertices

if v is not explored **then**
    add v to explored
    print [s..v]
    foreach vw $\in$ E **do**
        Depth_First_Search([s..v,w])

Depth-First Search for Explored Vertices





A	AB, AC
AC	AB | ACB, ACD, ACE
ACE	AB | ACB, ACD | ACED
ACED	AB | ACB, ACD
~~ACD~~	AB | ACB
ACB	AB | ACBE
~~ACBE~~	AB
~~AB~~	

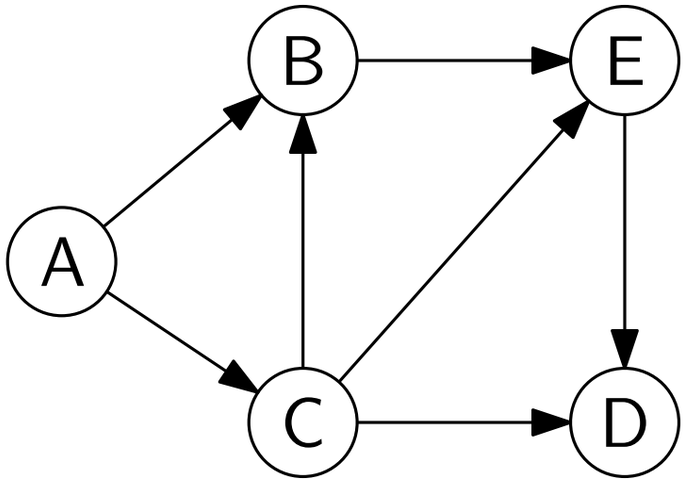
Depth-First Search for Path Tree

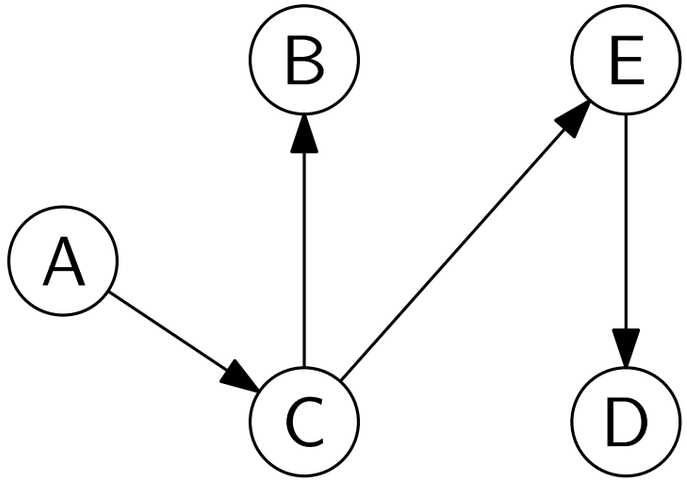
Algorithm: Depth_First_Search(v)

Input: v: vertex
Global: G = (V, E): DAG
parent: array of size n
explored: Boolean array of size n

if not explored[v] **then**
  explored[v] $\leftarrow$ True
  foreach vw $\in$ E **do**
    parent[w] $\leftarrow$ v
    Depth_First_Search(w)

Depth-First Search





A	AB, AC
AC	AB | CB, CD, CE
CE	AB | CB, CD | ED
ED	AB | CB, CD
~~CD~~	AB | CB
CB	AB | BE
~~BE~~	AB
~~AB~~	

Depth-First Search for a Goal Node

Algorithm: Depth_First_Search(v)

Input: v: vertex
Global: G = (V, E): DAG
parent: array of size n
explored: Boolean array of size n

```
if v is a goal vertex then
    return
if not explored[v] then
    explored[v] ← True
    foreach vw ∈ E do
        parent[w] ← v
        Depth_First_Search(w)
```

Depth First Search (DFS)

Algorithm: DF_Search(G, [s...,v], goal)

Input: G: graph
[s...,v]: path
goal: Boolean function on nodes

Output: path that extends [s...,v] to a goal node or failure if there is no such path.

```
if goal(v) then  return [s...,v]
foreach vw do
    result ← DF_Search(G, [s...,v,w], goal)
    if result ≠ failure then
        return result
return failure
```

- No multiple-path pruning.
- No explored set.

Search for a Solution in State Space

Search Problem

- States: Set of states
  - Start state
  - Goal states or Goal test: Does a state `s` satisfy goal property, `goal?(s)`
- Actions: `Actions(s)`, set of actions in state `s`, for every `s`
- State transition model
  - `NextState(s, a)`. In state `s`, if action `a` is done, what is the next state, for every `(s, a)`
  - `Successors(s)`
- Path cost: performance measure
  - Step cost: `Cost(s, a, s')`
  - Sum of step costs of along a path

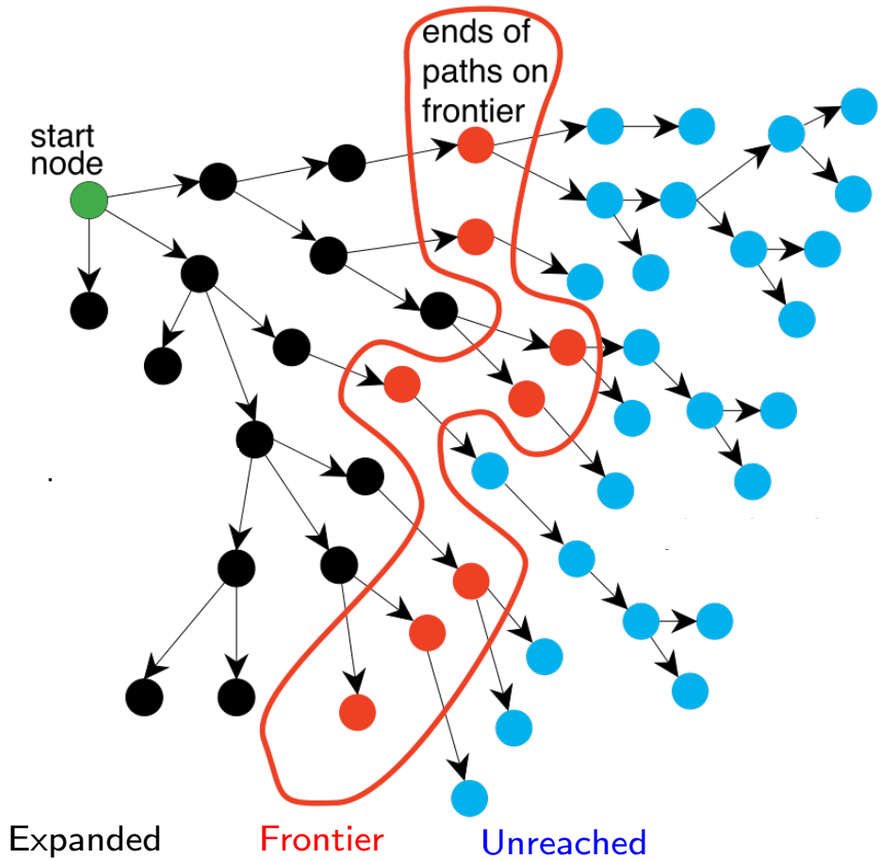
State Space and State Graph

- Set of all states reachable from start state by any sequence of actions
- State graph
  - nodes/vertices: states
  - edges: actions
  - edge weights: action costs
  - path: sequence of actions

Uninformed Search

- Uninformed search (Blind search)
  - Goal-agnostic!
  - Extend path (explore node) through any successor – no preference with regard to how nearer they are to goal nodes!
  - Test: goal node or non-goal node?
- Informed search (Heuristic search)
  - Goal-directed!
  - Nodes have some estimate (heuristic) about how nearer they are to a goal node.
  - Prefer to extend path (explore node) through successors "closer" to goal nodes.
- Search strategy
  - Order in which the paths are extended.

Expanded, Frontier, Unreached Nodes



- Expanded
  - explored nodes
  - closed nodes
  - interior nodes
- Frontier
  - discovered nodes
  - open nodes
  - tip nodes
- Unreached

Note

- Reached $\equiv$ Expanded $\cup$ Frontier
- Paths or nodes.

Whatever-First Search

Generic Search Algorithm with Multiple Path Pruning

Algorithm: State_Graph_Search (search_problem)

```
Explored ← {}
Frontier ← {(-,s)}
while Frontier is not empty do
    u, v ← select and remove an edge (u, v) from Frontier
    if v ∉ Explored then
        add v to Explored
        parent[v] ← u
        if v is a goal node then  return solution
        foreach successor w of v do
            add (v, w) to Frontier
```

Frontier and Explored Data Structures

- Frontier: Bag data structure, with operations
  - `add(item, bag)`
  - `remove(bag): Item`
  - `empty?(bag): Boolean`
- Search strategy: order in which items are added and removed from the bag (frontier) – Bag can be
  - Queue: First In First Out – Breadth-First Search
  - Stack: last in first out – Depth-First Search
  - Priority Queue (Heap): minimum cost first – Priority-First Search
- Explored: Set, implemented as a Boolean array

Generic Search and Other Algorithms

- Depth-First Search (DFS)
- Bread-First Search (BFS)
- Priority-First Search (PFS)
  - Uniform-Cost-First Search(UCS): $d[v] = f[v]$
  - Best-First-Search: $d[v] = g[v]$
  - A* Search: $d[v] = f[v] + g[v]$

Measuring Search Performance

- Completeness: If there is a path to a goal node, is the algorithm guaranteed to find a path?
- Optimality: Does the algorithm find the optimal path?
- Time complexity: How long does it take to find a path?
- Space complexity: How much memory is needed to perform the search?

Depth-First-Search

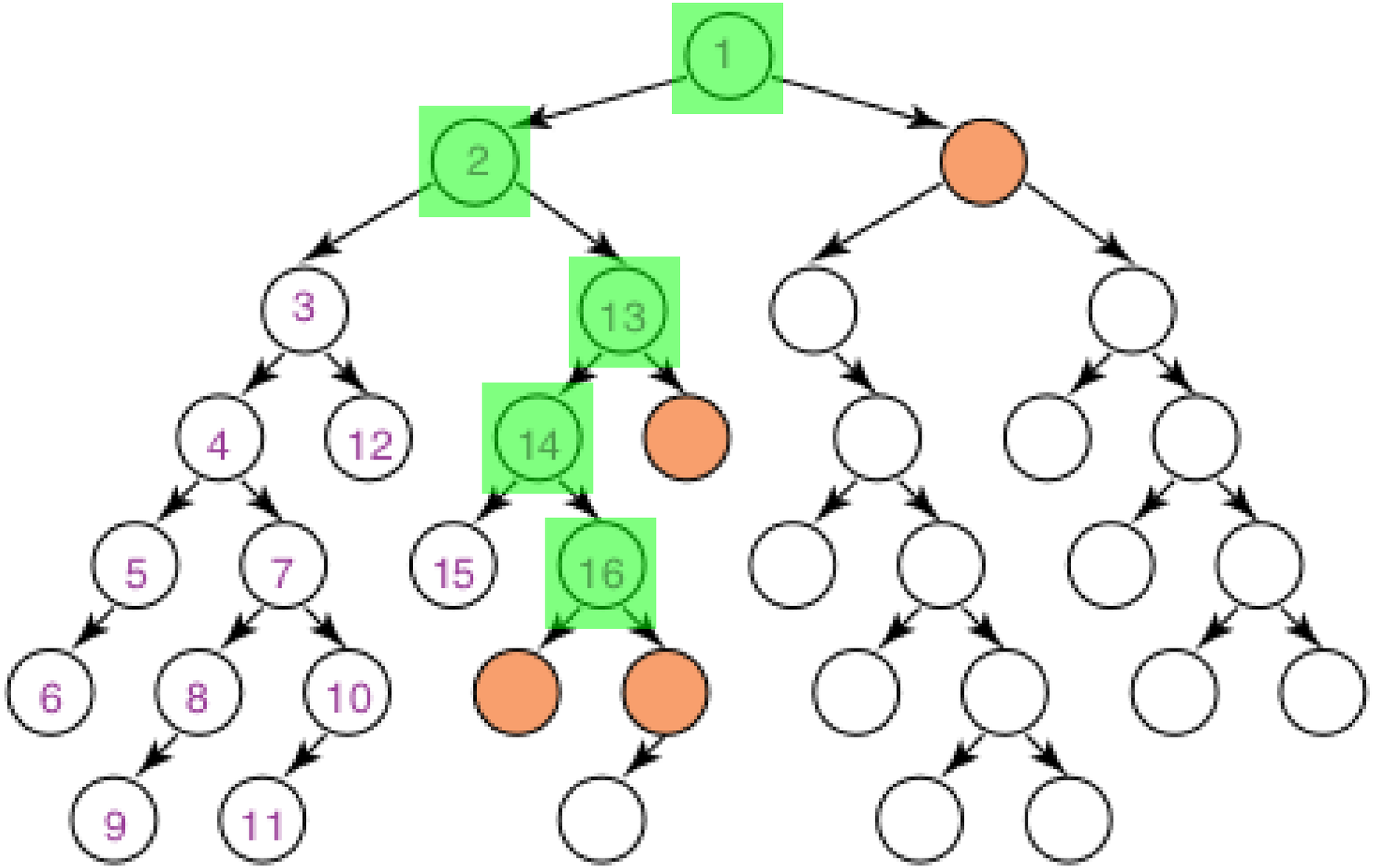
Depth-First = Last-In First Out

```
Algorithm: Depth_First_Search (search_problem)

Explored ← {}
clock ← 0; clock[s] ← clock
Frontier ← {(-,s)}
while Frontier is not empty do
    u, v ← remove the edge (u, v) with the maximum clock[v] from Frontier
    if v ∉ Explored then
        add v to Explored
        parent[v] ← u
        clock ← clock + 1
        foreach successor w of v do
            add (v, w) to Frontier; clock[w] ← clock
```

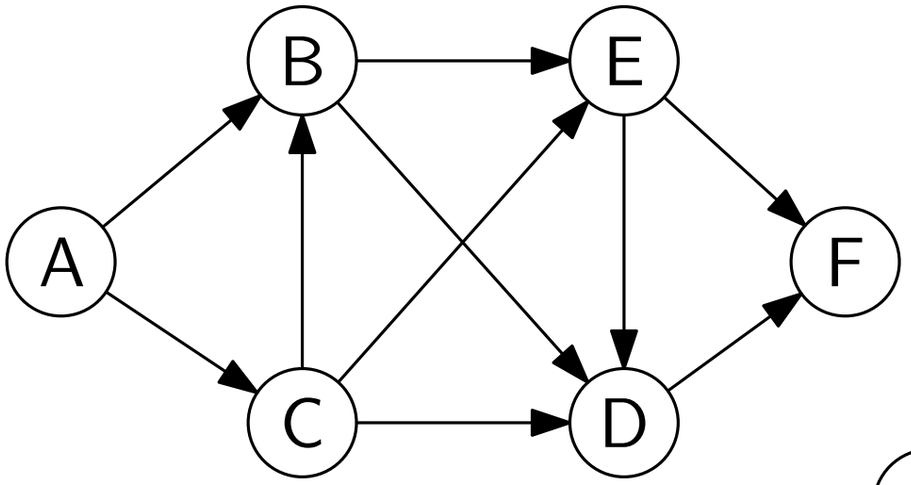
Depth-First Search Using Stack

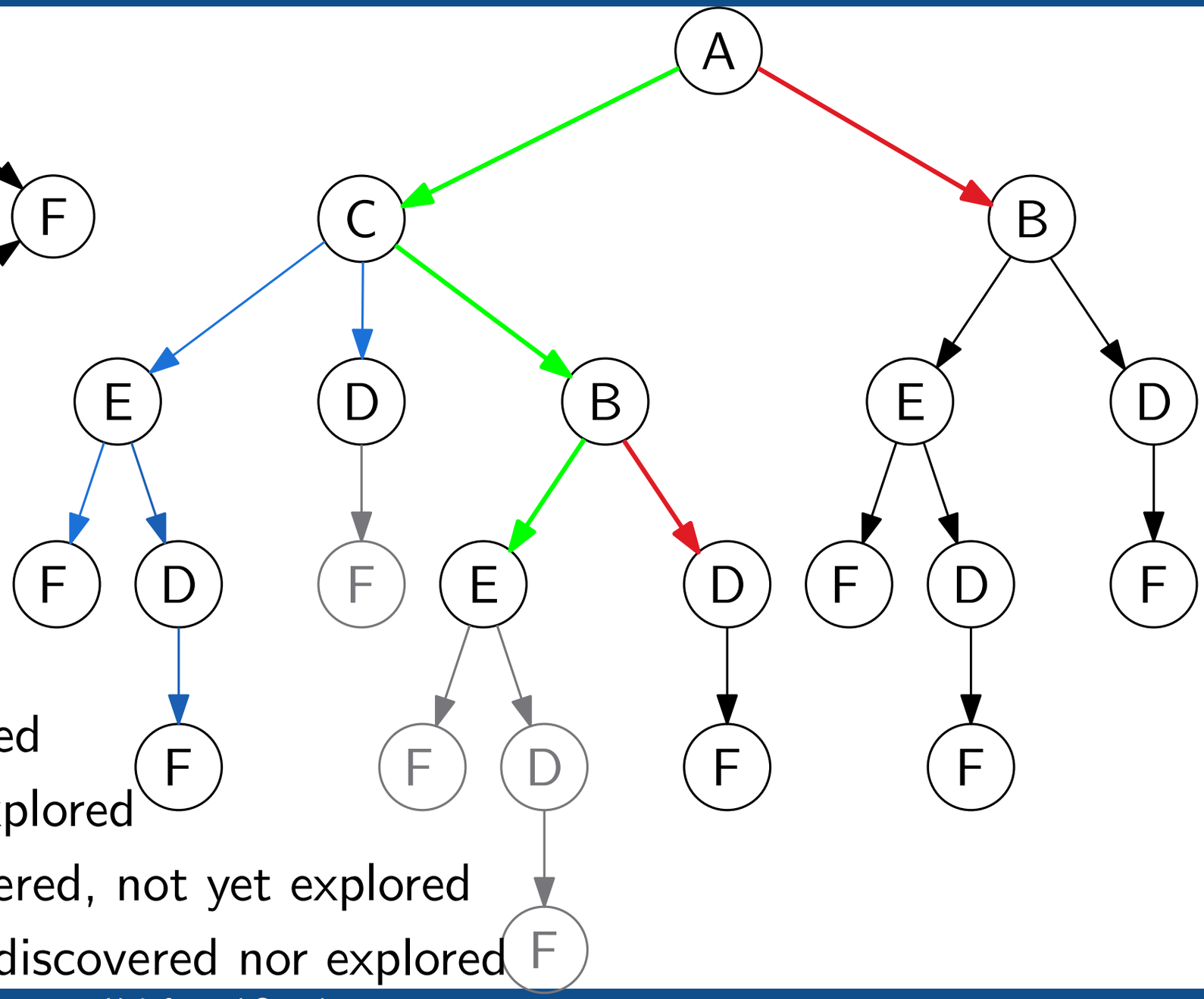
```
Algorithm: Depth_First_Search (search_problem)

Explored ← {}
push (−, s) to Stack
while Stack is not empty do
    u, v ← pop (u, v) from Stack
    if v ∉ Explored then
        add v to Explored
        parent[v] ← u
        if v is a goal node then  return solution
        foreach successor w of v do
            push (v, w) to Stack
```

DFS Order and Frontier



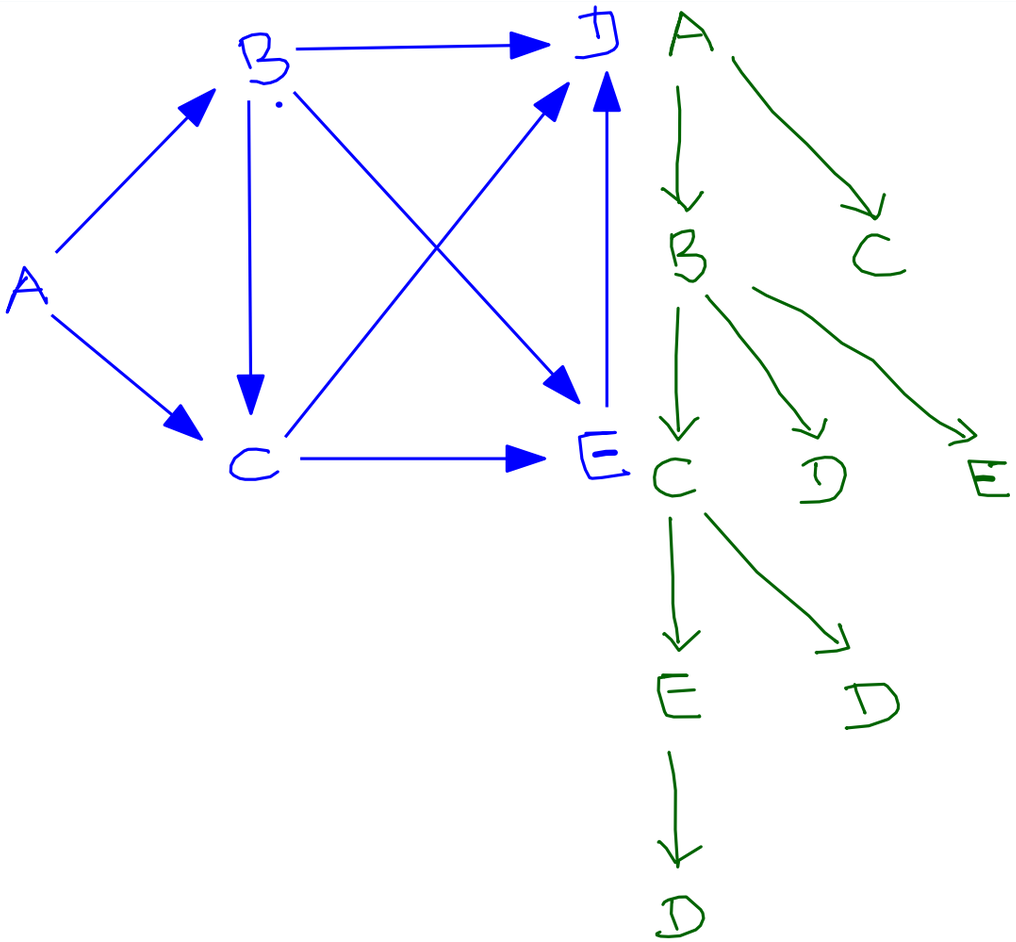
DFS nodes: Finished, Active, Pending, Future





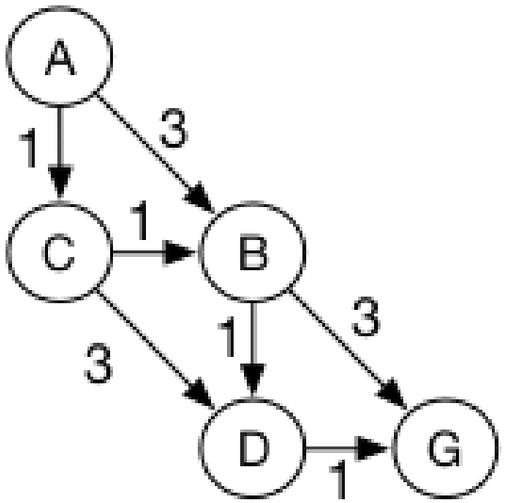
- Finished nodes: explored
- Active nodes: being explored
- Pending nodes: discovered, not yet explored
- Future nodes: neither discovered nor explored

DFS Paths are Long



- Remove the path (of depth k) from the top of the stack.
- Expand the path and add the expanded paths (of depth $k+1$) to the top of the stack.
- Now a path (of depth $k+1$) is on the top of the stack.
- Paths tend to be deeper.

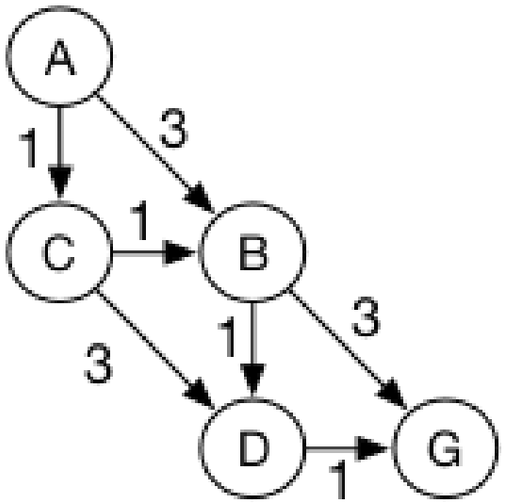
DFS Trace



E F

0 -A0

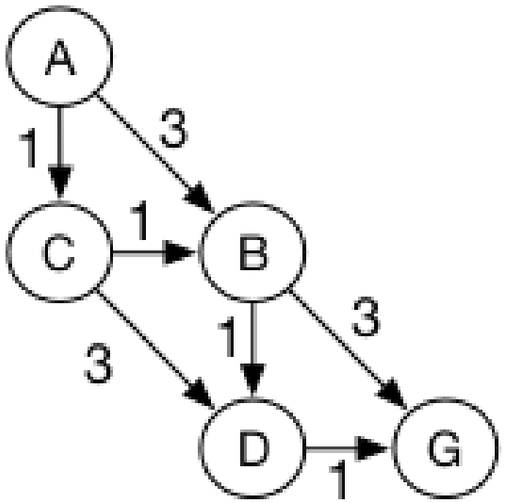
DFS Trace



	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1

A

DFS Trace

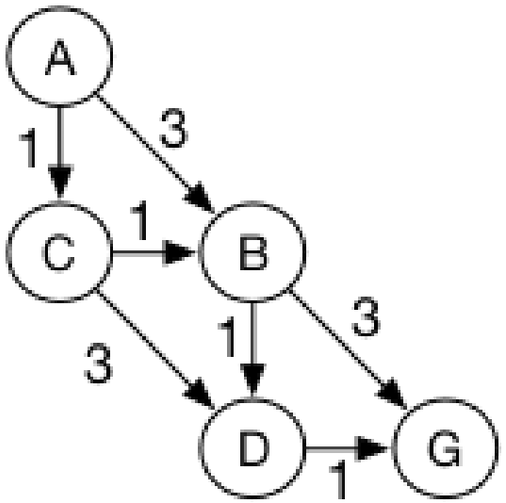


	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2

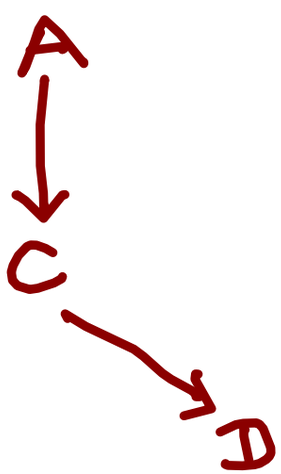
A

C

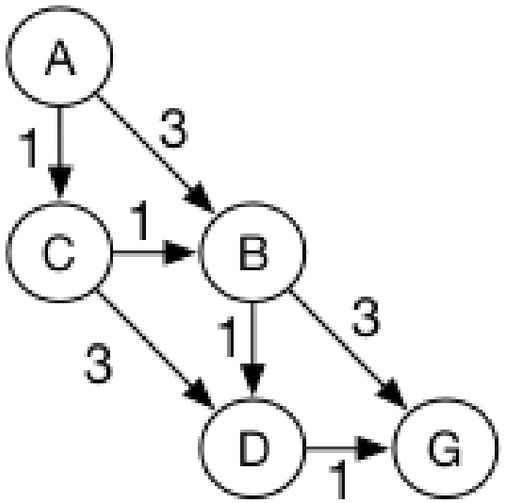
DFS Trace



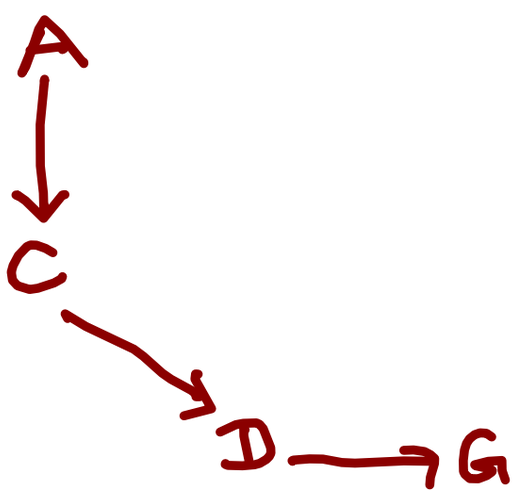
	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2
3	CD_2	AB_1, CB_2, DG_3



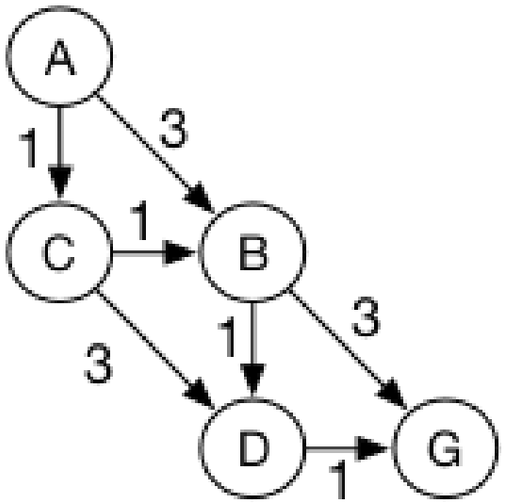
DFS Trace



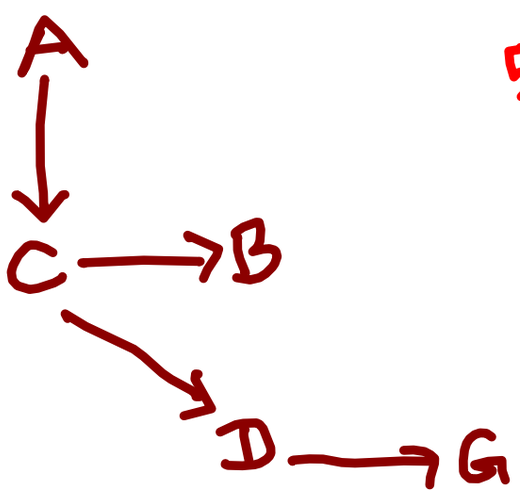
	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2
3	CD_2	AB_1, CB_2, DG_3
4	DG_3	AB_1, CB_2



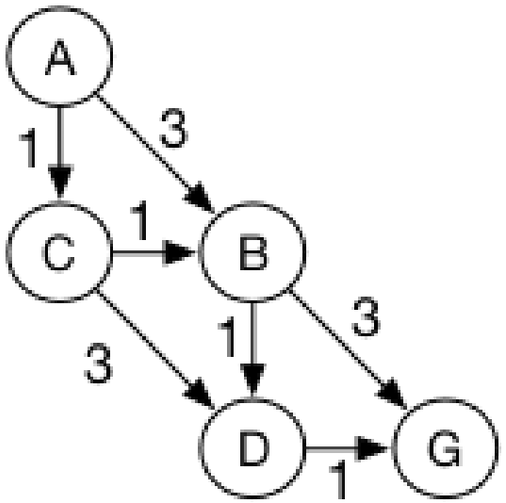
DFS Trace



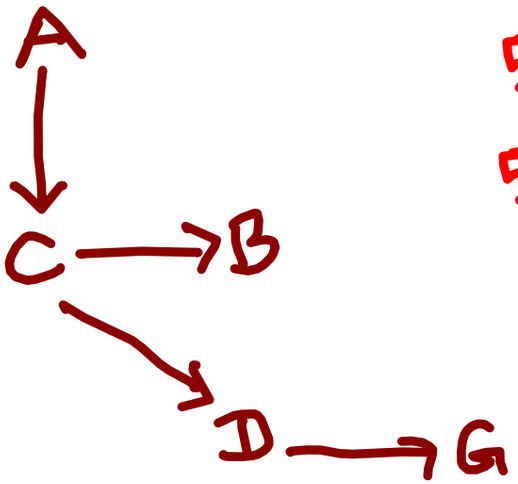
	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2
3	CD_2	AB_1, CB_2, DG_3
4	DG_3	AB_1, CB_2
5	CB_2	AB_1, BD_5, BG_5



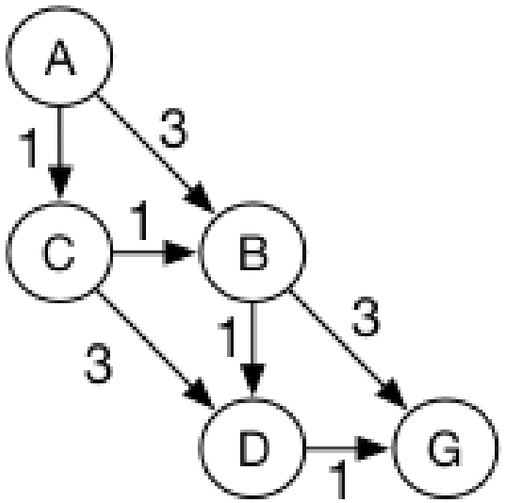
DFS Trace



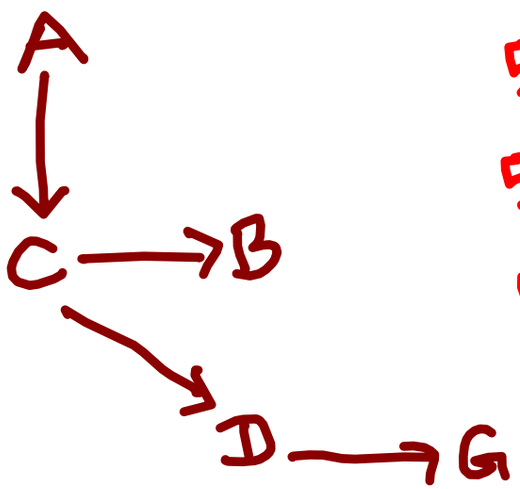
	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2
3	CD_2	AB_1, CB_2, DG_3
4	DG_3	AB_1, CB_2
5	CB_2	AB_1, BD_5, BG_5
5	$\cancel{BG_5}$	AB_1, BD_5



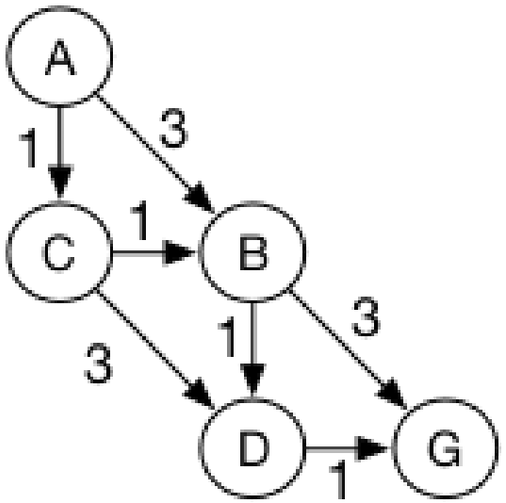
DFS Trace



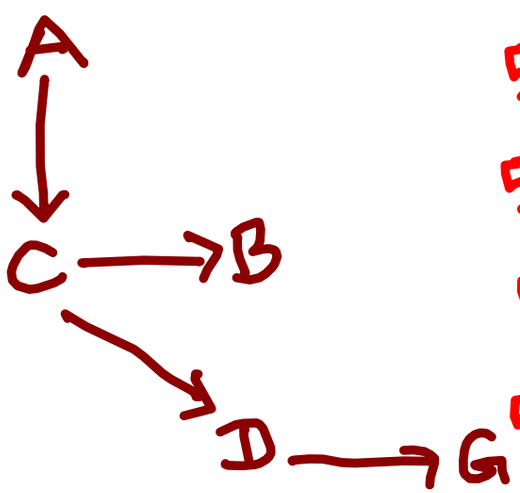
	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2
3	CD_2	AB_1, CB_2, DG_3
4	DG_3	AB_1, CB_2
5	CB_2	AB_1, BD_5, BG_5
5	$\cancel{BG_5}$	AB_1, BD_5
5	$\cancel{BD_5}$	AB_1



DFS Trace



	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AC_1	AB_1, CB_2, CD_2
3	CD_2	AB_1, CB_2, DG_3
4	DG_3	AB_1, CB_2
5	CB_2	AB_1, BD_5, BG_5
5	$\cancel{BG_5}$	AB_1, BD_5
5	$\cancel{BD_5}$	AB_1
5	$\cancel{AB_1}$	



DFS Performance

Stores just one path from start node to frontier node
b branching factor, # of successors to each node

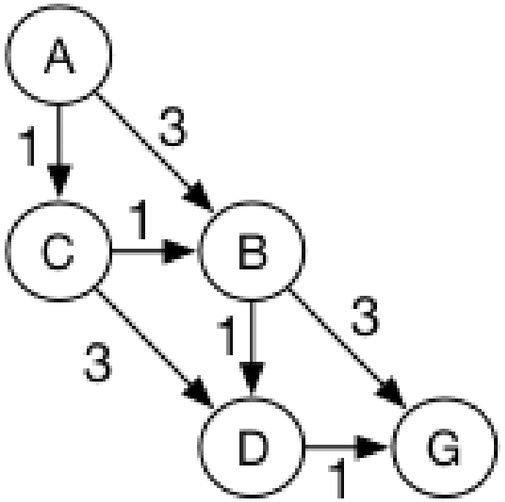
- Time complexity
  - m, maximum depth of any node; d, depth of goal node, $m >> d$
  - Generates $O(b^m)$ nodes
- Space complexity: DFS iterative verison
  - Only a single path from root to leaf (of length m)
  - Store b unexpanded successors of each node in the path in stack
  - $O(bm)$
- Space complexity: DFS recursive version
  - Only a single path from root to leaf (of length m)
  - Store only one successor of each node in the stack
  - $O(m)$

Breadth-First-Search

Breadth-First Search (BFS) Frontier

- Frontier = Queue (FIFO)
  - Add successors to the rear of the queue
  - Remove nodes from the front of the queue
  - Order: Removes nodes from the frontier in the order they were added to the frontier.

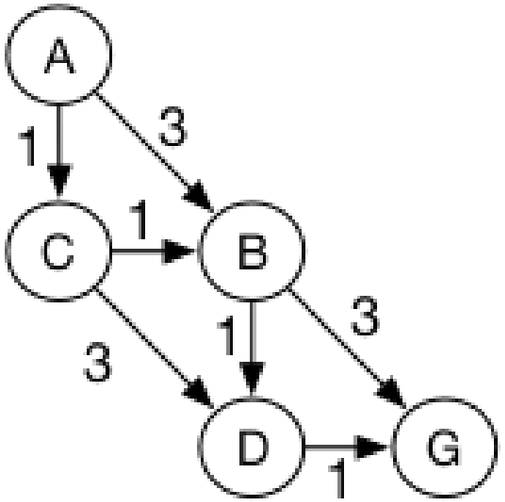
BFS Trace



d E F

—A.

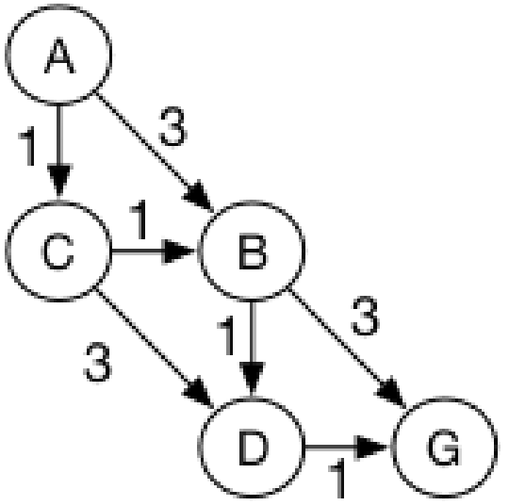
BFS Trace



d	E	F
		$-A_0$
$-A_0$	AB_1, AC_1	

A

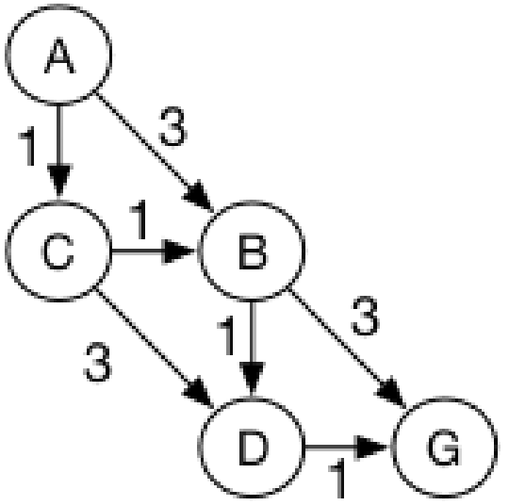
BFS Trace



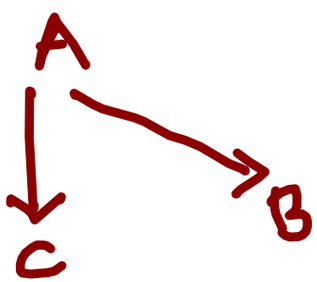
d	E	F
		$-A_0$
$-A_0$		AB_1, AC_1
AB_1		AC_1, BD_2, BG_2

$A \to B$

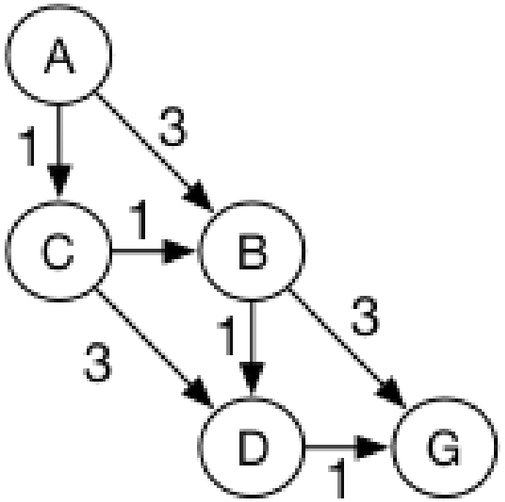
BFS Trace



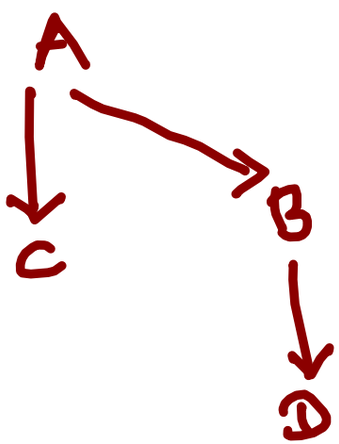
d	E	F
		$-A_0$
$-A_0$		AB_1, AC_1
AB_1		AC_1, BD_2, BG_2
AC_1		BD_2, BG_2, CB_2, CD_2



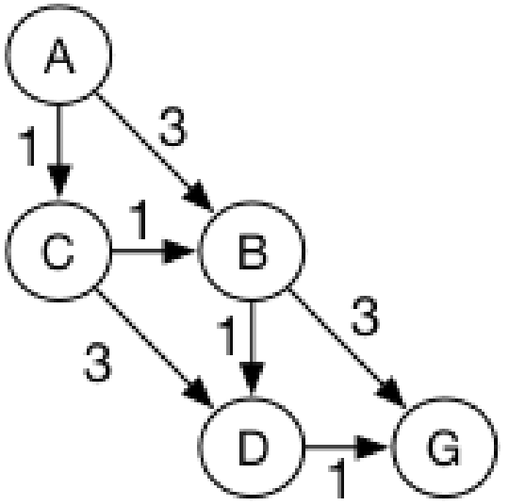
BFS Trace

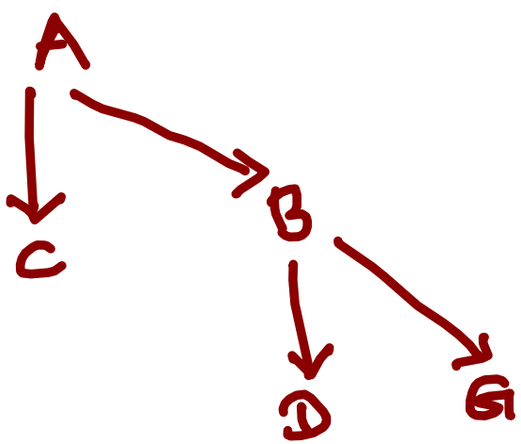


d	E	F
		$-A_0$
$-A_0$	AB_1, AC_1	
AB_1	AC_1, BD_2, BG_2	
AC_1	BD_2, BG_2, CB_2, CD_2	
BD_2	BG_2, CB_2, CD_2, DG_3	



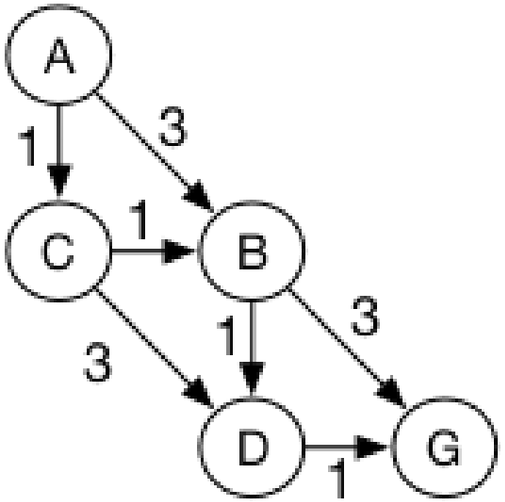
BFS Trace

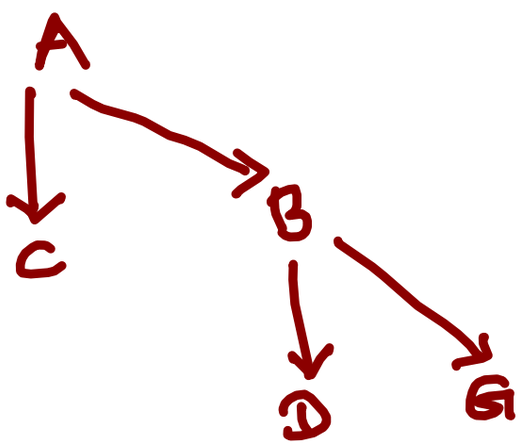




d	E	F
		$-A_0$
$-A_0$		AB_1, AC_1
AB_1		AC_1, BD_2, BG_2
AC_1		BD_2, BG_2, CB_2, CD_2
BD_2		BG_2, CB_2, CD_2, DG_3
BG_2		CB_2, CD_2, DG_3

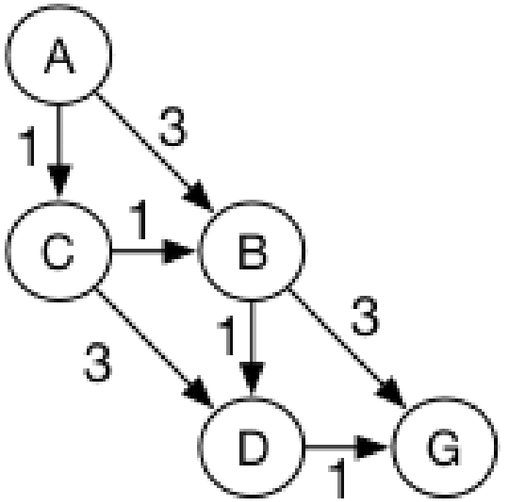
BFS Trace

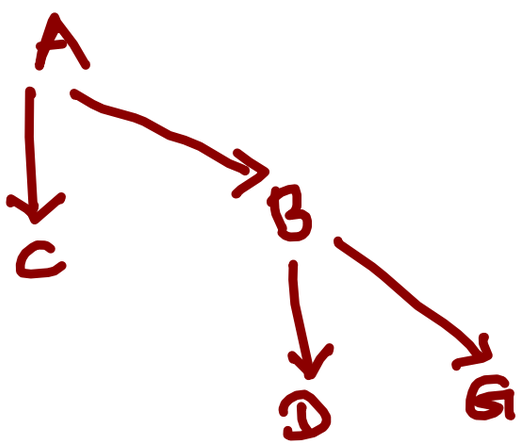




d	E	F
		$-A_0$
$-A_0$		AB_1, AC_1
AB_1		AC_1, BD_2, BG_2
AC_1		BD_2, BG_2, CB_2, CD_2
BD_2		BG_2, CB_2, CD_2, DG_3
BG_2		CB_2, CD_2, DG_3
$\cancel{CB_2}$		CD_2, DG_3

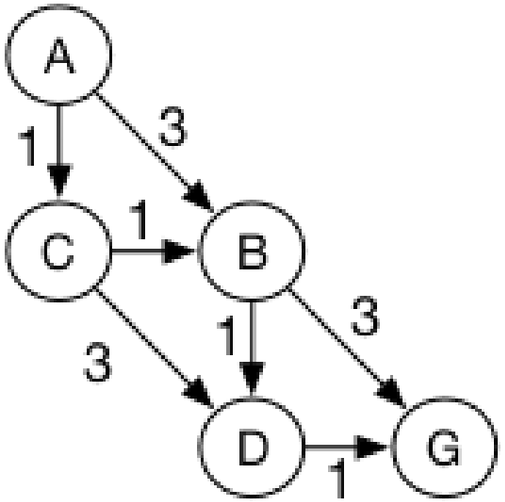
BFS Trace

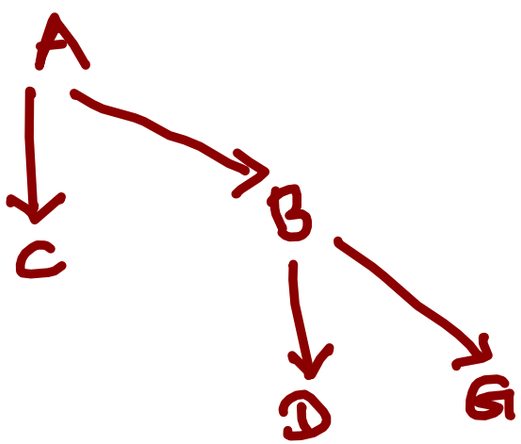




d	E	F
		$-A_0$
$-A_0$		AB_1, AC_1
AB_1		AC_1, BD_2, BG_2
AC_1		BD_2, BG_2, CB_2, CD_2
BD_2		BG_2, CB_2, CD_2, DG_3
BG_2		CB_2, CD_2, DG_3
$\cancel{CB_2}$		CD_2, DG_3
$\cancel{CD_2}$		DG_3

BFS Trace





d	E	F
		A_0
A_0		AB_1, AC_1
AB_1		AC_1, BD_2, BG_2
AC_1		BD_2, BG_2, CB_2, CD_2
BD_2		BG_2, CB_2, CD_2, DG_3
BG_2		CB_2, CD_2, DG_3
$\cancel{CB_2}$		CD_2, DG_3
$\cancel{CD_2}$		DG_3
$\cancel{DG_3}$		

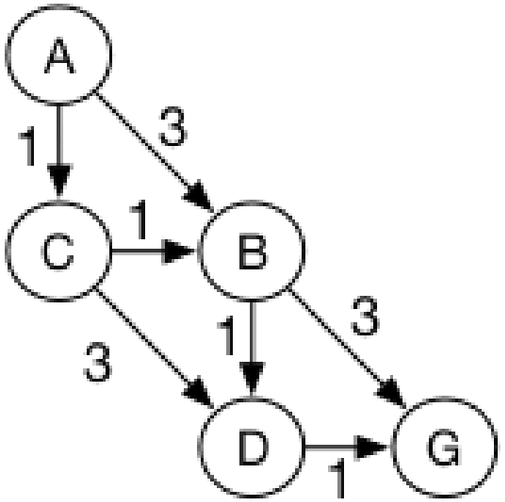
Breadth-First Search Using Queue

```
Algorithm: Breadth_First_Search (search_problem)
Explored ← {}
add (−, s) to Queue
while Queue is not empty do
    u, v ← remove (u, v) from Queue
    if v ∉ Explored then
        add v to Explored
        parent[v] ← u
        if v is a goal node then  return solution
        foreach successor w of v do
            add (v, w) to Queue
```

Breadth-First Search Using Distance Data Structure

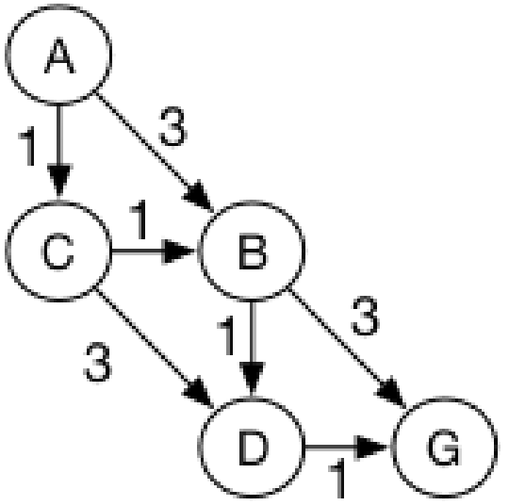
```
Algorithm: Breadth_First_Search (search_problem)
foreach v do  d[v] ← ∞  //  d[v] = ∞ ≡ v ∉ E
d[s] ← 0
add (−, s) to Queue
while Queue is not empty do
    u, v ← remove (u, v) from Queue
    if d[v] = ∞ then //  v ∉ E
        d[v] ← d[u] + 1
        parent[v] ← u
        if v is a goal node then  return solution
        foreach successor w of v do
            add (v, w) to Queue
```

BFS Trace



d	E	F
0		-A.

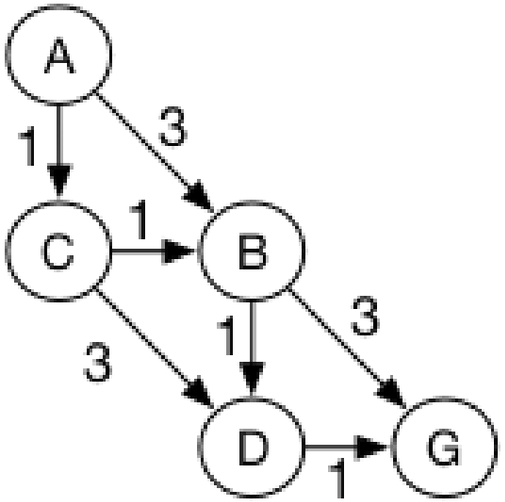
BFS Trace



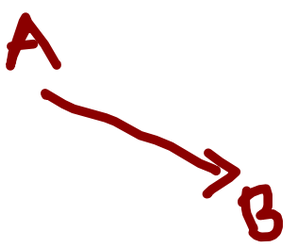
d	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1

A

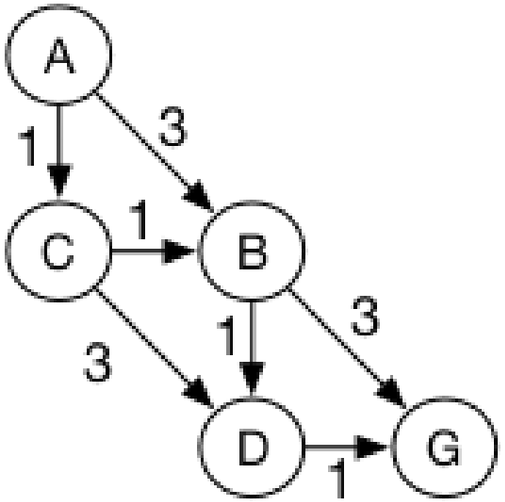
BFS Trace



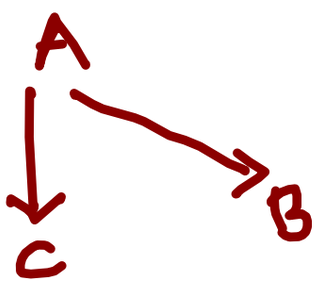
d	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AB_1	AC_1, BD_2, BG_2



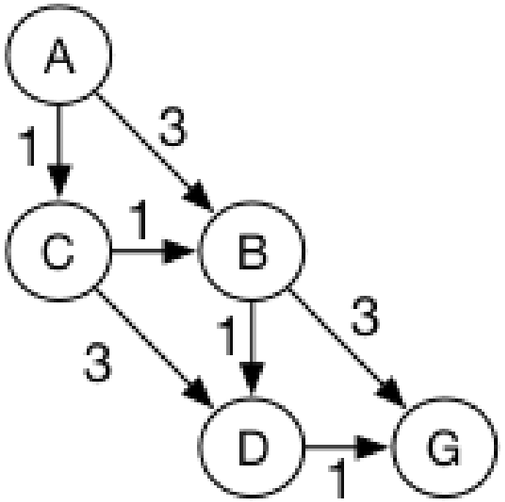
BFS Trace



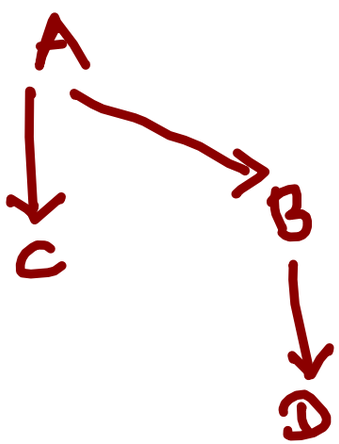
d	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AB_1	AC_1, BD_2, BG_2
2	AC_1	$BD_2, BG_2, C\cancel{B}_2, C\cancel{D}_2$



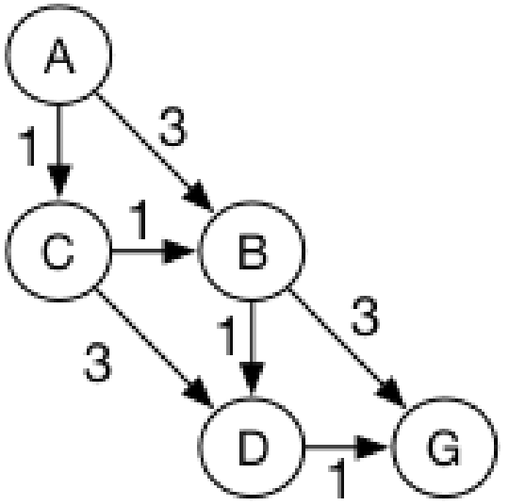
BFS Trace



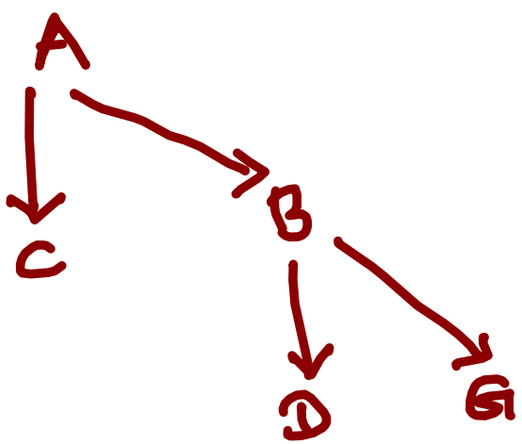
d	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AB_1	AC_1, BD_2, BG_2
2	AC_1	$BD_2, BG_2, C\cancel{B}_2, C\cancel{D}_2$
3	BD_2	$BG_2, D\cancel{G}_3$



BFS Trace



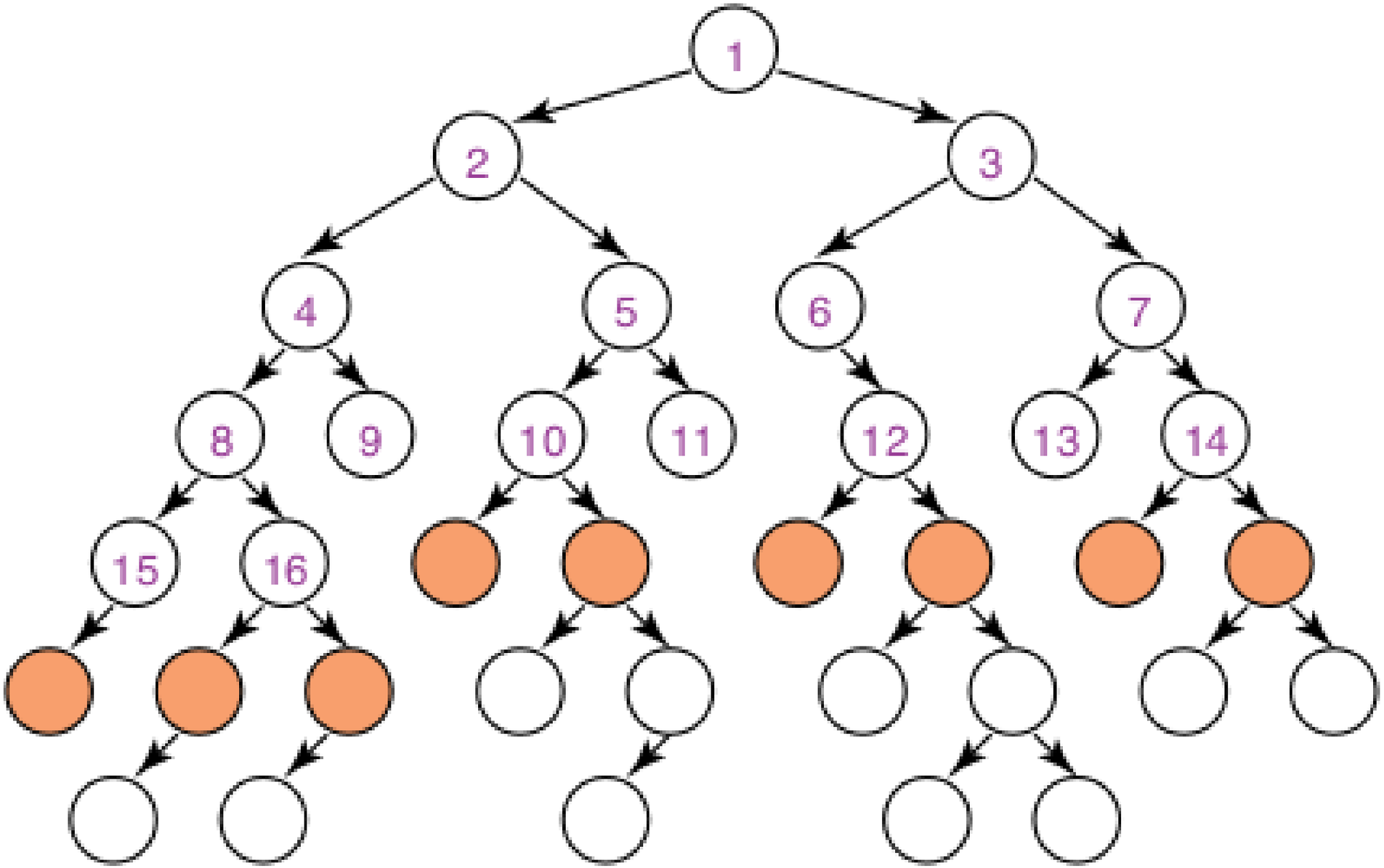
d	E	F
0		$-A_0$
1	$-A_0$	AB_1, AC_1
2	AB_1	AC_1, BD_2, BG_2
2	AC_1	$BD_2, BG_2, C\cancel{B}_2, C\cancel{D}_2$
3	BD_2	$BG_2, D\cancel{G}_3$
3	BG_2	



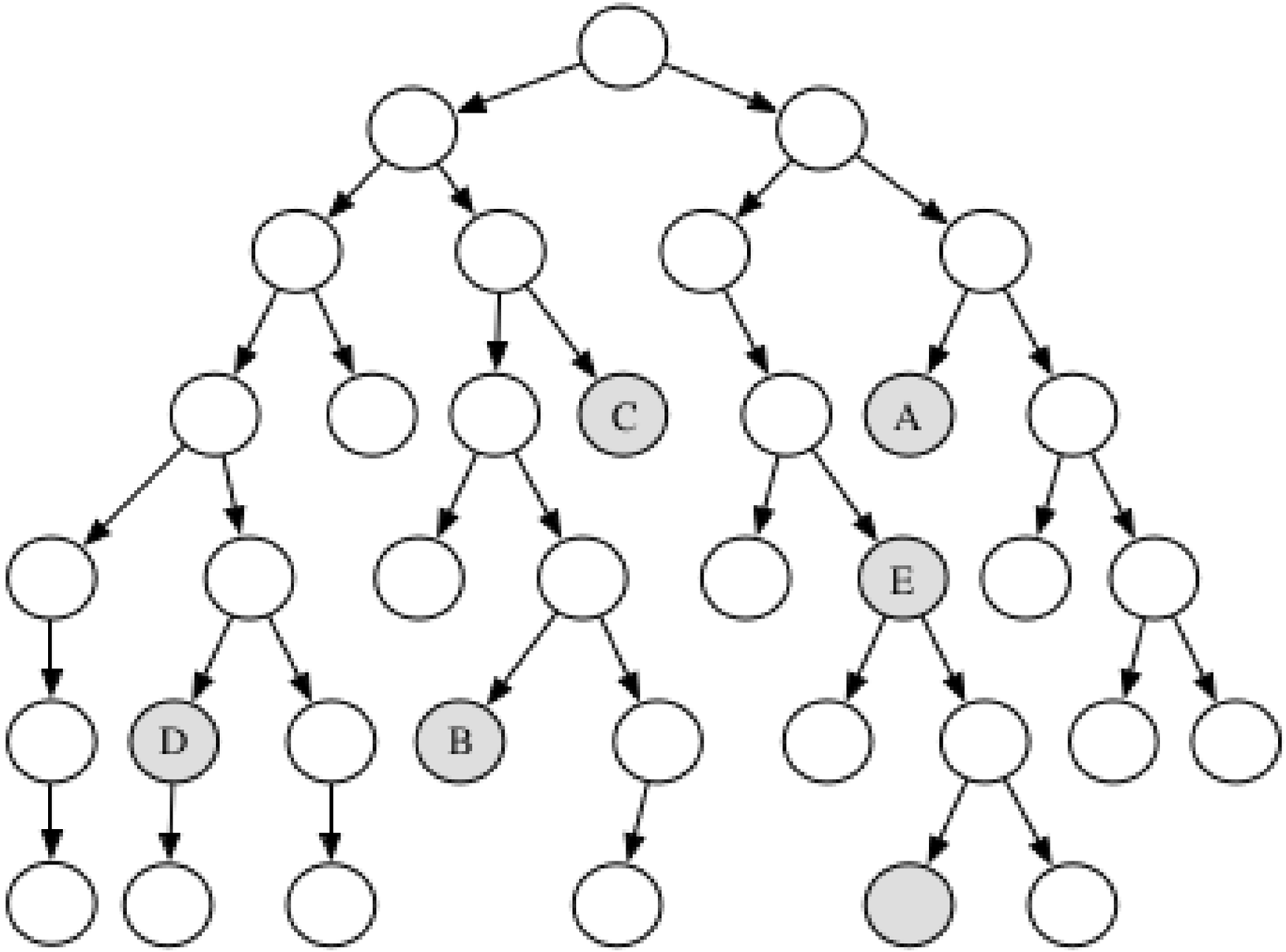
Breadth-First Search: Prune at Entry

```
Algorithm: Breadth_First_Search (search_problem)
foreach v do  d[v] ← ∞  //  d[v] = ∞ ≡ v ∉ E
d[s] ← 0
add (−, s) to Queue
while Queue is not empty do
    u, v ← remove (u, v) from Queue
    if v is a goal node then  return solution
    foreach successor w of v do
        if d[w] = ∞ then //  d[v] = ∞ ≡ v ∉ (E ∪ F)
            d[w] ← d[v] + 1
            parent[w] ← v
            add (v, w) to Queue
```

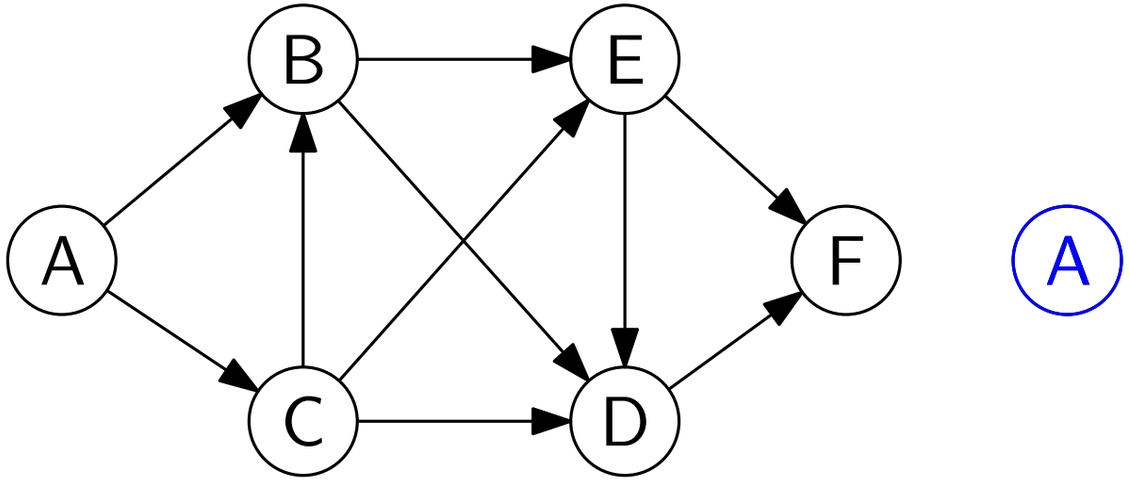
BFS Frontier



Which Shaded Node BFS Finds First?

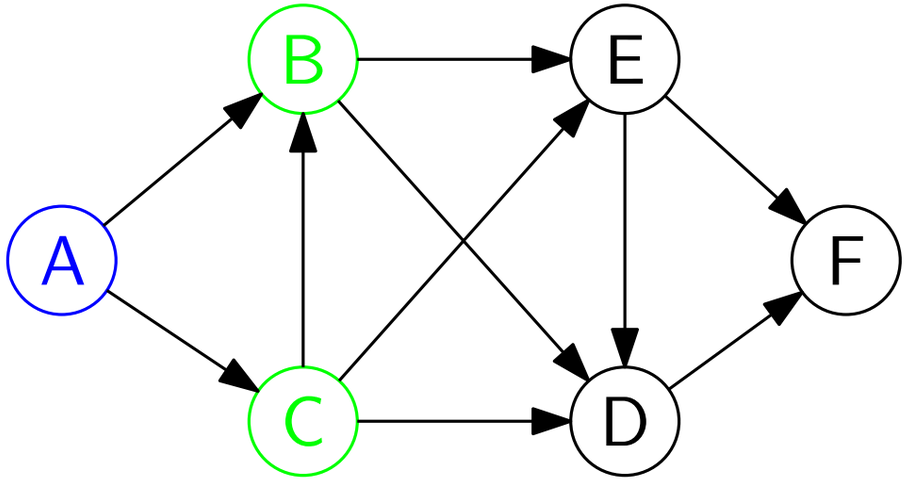


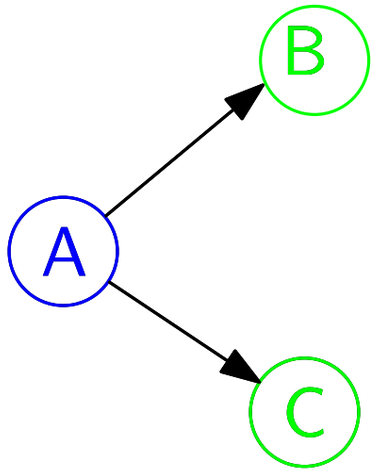
BFS Frontier Spreads Like a Wave



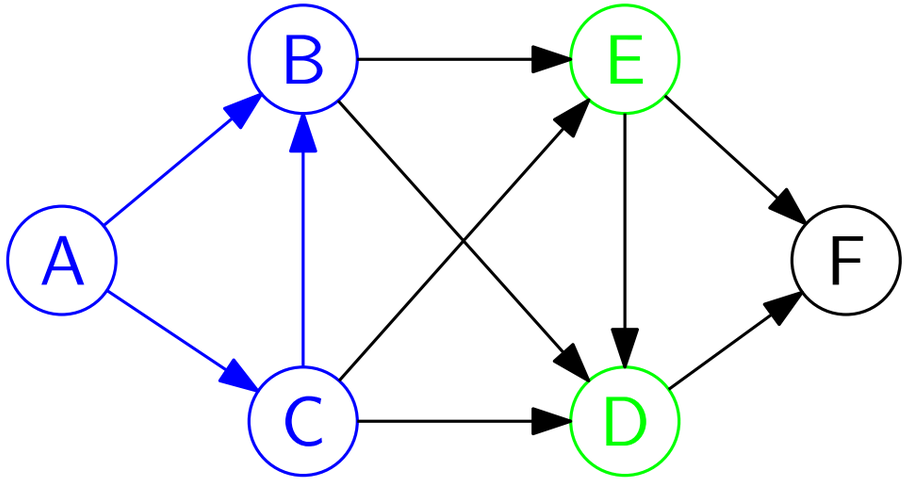
A

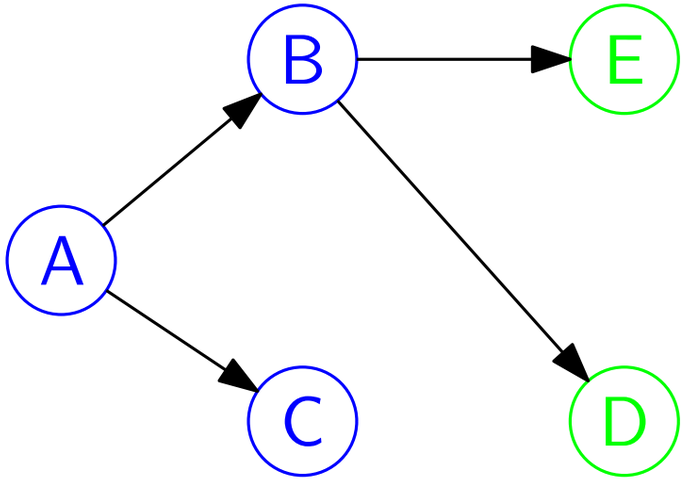
BFS Frontier Spreads Like a Wave



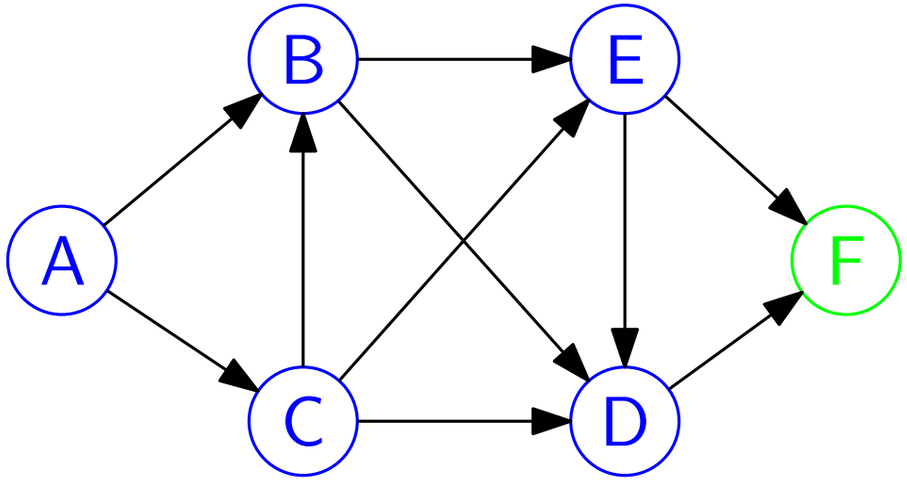


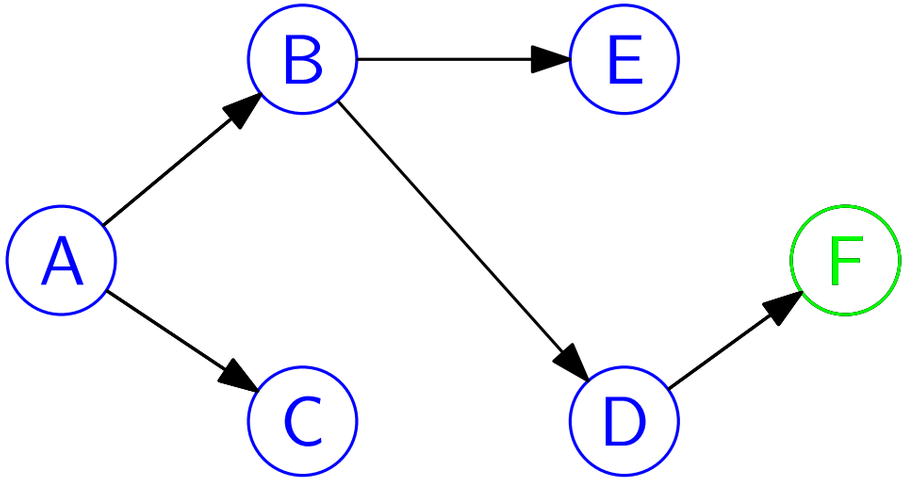
BFS Frontier Spreads Like a Wave





BFS Frontier Spreads Like a Wave





Breadth-First Search Properties

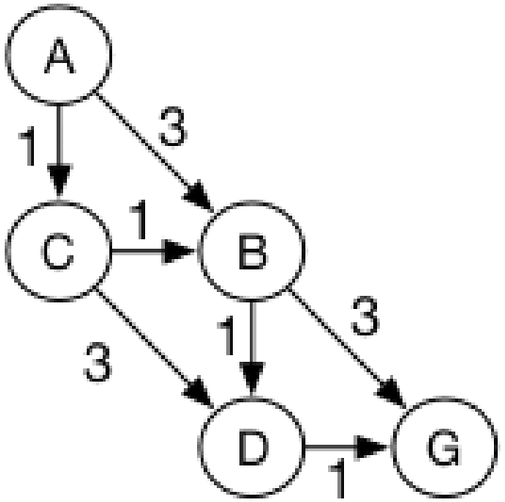
- Order: Removes frontier nodes in the order they were added to the frontier.
- Removes/adds frontier nodes in the hop-distance order.
  - A node's successors are one hop farther from it.
  - Suppose all nodes at distance k are in the Queue.
  - As they are removed, their successors at distance $k+1$ are added to the Queue.
  - All nodes at distance $k+1$ are in the Queue.
- Explores paths in the hop-distance order.
  - Start node
  - Nodes 1-hop away
  - Nodes 2-hops away ...

BFS Performance

- Complete
  - Finds the shortest solution.
- Optimal
  - If all actions/edges have the same cost.
  - If path cost is a non-decreasing function of depth of the node.
- Time complexity
  - Assume a uniform b-ary tree $b + b^2 + b^3 + \ldots + b^d = O(b^d)$
- Space complexity
  - Explored set: $O(b^{d-1})$
  - Frontier $O(b^d)$
  - Dominated by Frontier
- Memory requirements are a bigger problem than the execution time
- Exponential-complexity search problems cannot be solved by uninformed methods for any but the smallest instances.

Priorty-First-Search

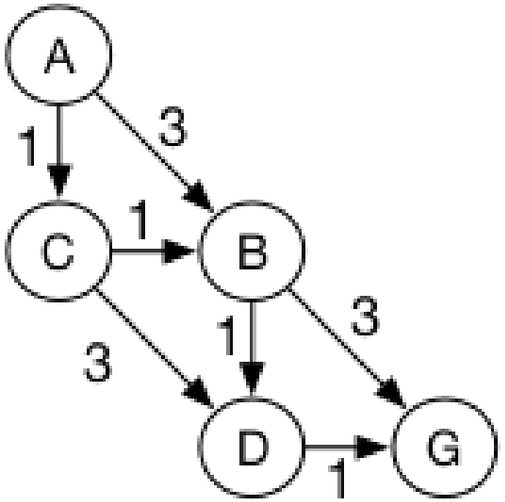
PFS Example



E F

- A o

PFS Example



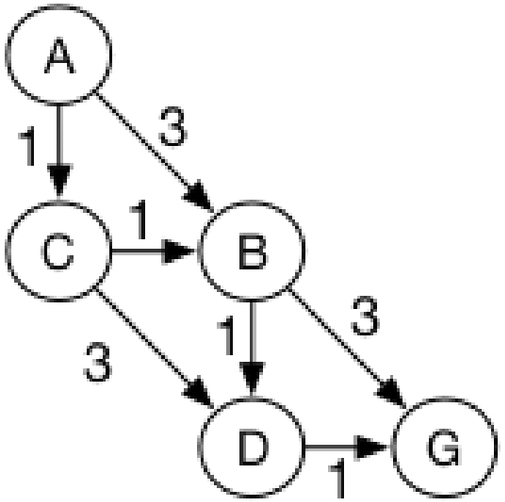
E F

$-A_0$

$-A_0$ AB_3, AC_1

A

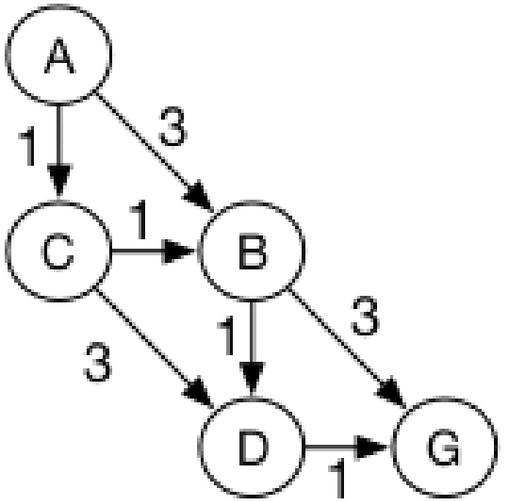
PFS Example



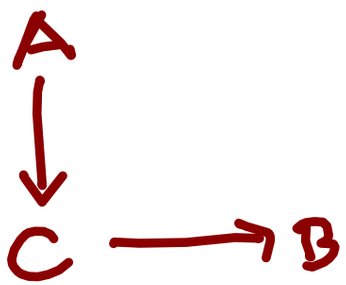
E	F
	$-A_0$
$-A_0$	AB_3, AC_1
AC_1	$\cancel{AB_3}$, CB_2, CD_4

$A \rightarrow C$

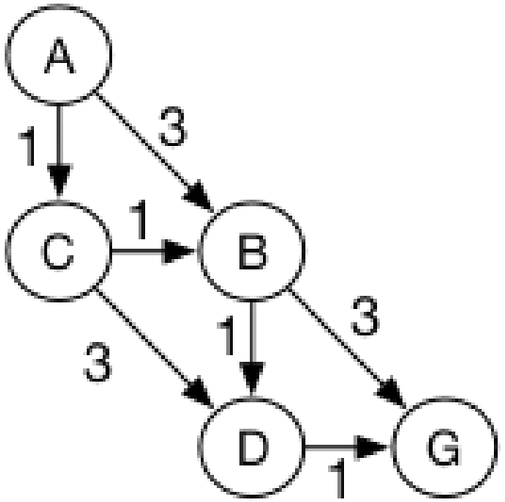
PFS Example

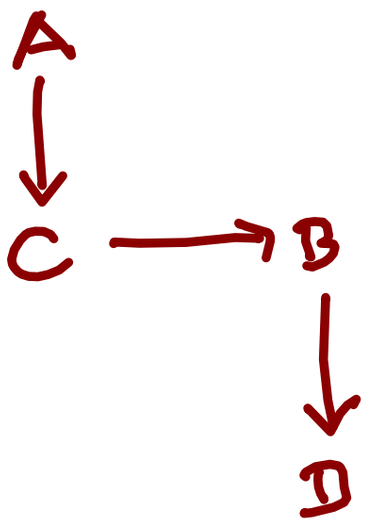


E	F
	$-A_0$
$-A_0$	AB_3, AC_1
AC_1	$\cancel{AB_3}$, CB_2, CD_4
CB_2	$\cancel{CD_4}$, BD_3, BG_5



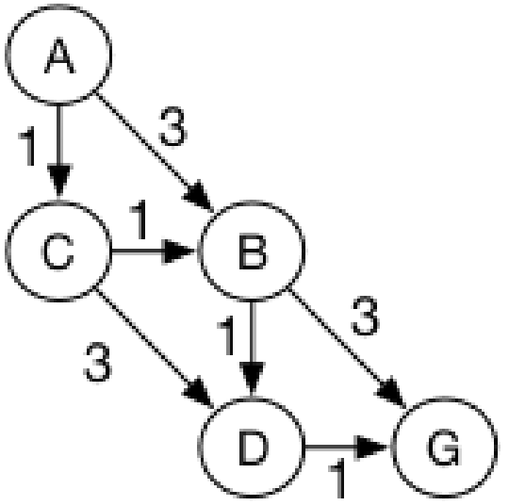
PFS Example

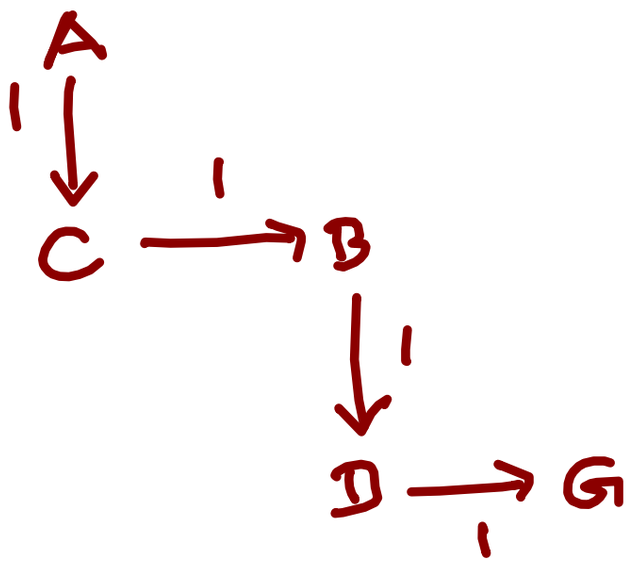




E	F
	$-A_0$
$-A_0$	AB_3, AC_1
AC_1	$\cancel{AB_3}$, CB_2, CD_4
CB_2	$\cancel{CD_4}$, BD_3, BG_5
BD_3	$\cancel{BG_5}$, DG_4

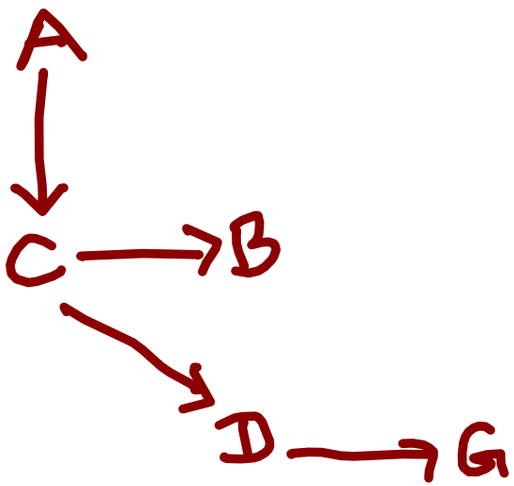
PFS Example



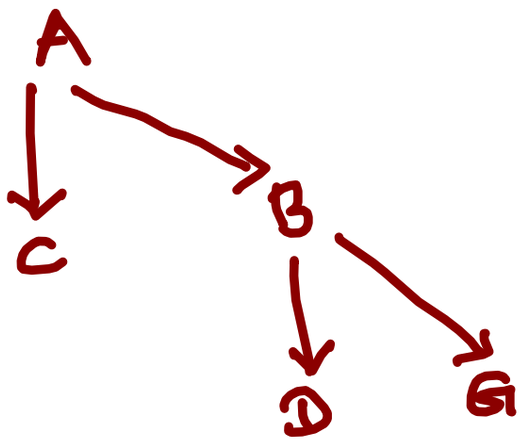


E	F
	$-A_0$
$-A_0$	AB_3, AC_1
AC_1	$\cancel{AB_3}$, CB_2, CD_4
CB_2	$\cancel{CD_4}$, BD_3, BG_5
BD_3	$\cancel{BG_5}$, DG_4
DG_4	

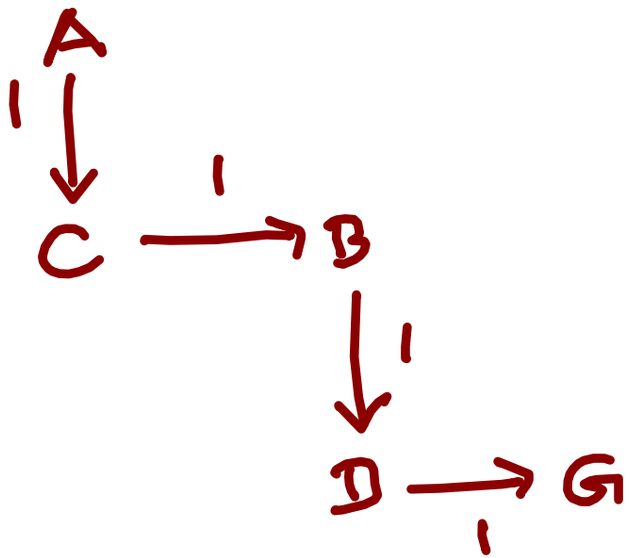
DFS, BFS, PFS Path Trees



DFS



BFS



PFS

Generic Search Algorithm with Multiple Path Pruning

Algorithm: State_Graph_Search (search_problem)

```
Explored ← {}
Frontier ← {(-,s)}
while Frontier is not empty do
    u, v ← select and remove an edge (u, v) from Frontier
    if v ∉ Explored then
        add v to Explored
        parent[v] ← u
        if v is a goal node then  return solution
        foreach successor w of v do
            add (v, w) to Frontier
```

Priority-First Search with Multiple Path Pruning

```
Algorithm: Priority_First_Search (search_problem)
Explored ← {}
add (−, s) to PriorityQ
while PriorityQ is not empty do
    u, v ← remove edge (u, v) with highest priority from PriorityQ
    if v ∉ Explored then
        add v to Explored
        parent[v] ← u
        if v is a goal node then  return solution
        foreach successor w of v do
            add (v, w) to PriorityQ
```

Priority-First Search: Distance Data Structure

```
Algorithm: Priority_First_Search (search_problem)
Explored ← {}
add (−, s) to PriorityQ
d[s] ← 0
while PriorityQ is not empty do
    u, v ← remove edge (u, v) with minimum d[u] + c(u, v) in PriorityQ
    if v ∉ E then -- Explore v
        add v to Explored
        d[v] ← d[u] + c(u,v)
        parent[v] ← u
        foreach successor w of v do
            add (v, w) to PriorityQ
```

Priority-First Search: Explored Using Distance

```
Algorithm: Priority_First_Search (search_problem)
foreach v do  d[v] ← ∞
d[s] ← 0
add (−, s) to PriorityQ
while PriortyQ is not empty do
    u, v ← remove edge (u, v) with minimum d[u] + c(u, v) in PriorityQ
    if d[u] + c(u,v) < d[v] then
        d[v] ← d[u] + c(u,v)
        parent[v] ← u
        foreach successor w of v do
            add (v, w) to PriorityQ
```

Priority-First Search Algorithm: Prune at Entry

```
Algorithm: Priority_First_Search (search_problem)
foreach v do  d[v] ← ∞
d[s] ← 0
add (d[s], -, s) to PriorityQ
while PriortyQ is not empty do
    u, v ← remove edge (u,v) with minimum d[v] in PriortyQ
    foreach successor w of v do
        if d[v] + c(v,w) < d[w] then
            d[w] ← d[v] + c(v,w)
            parent[w] ← v
            if v ∈ PriorityQ then
                update (v,w, d[w]) in PriorityQ
            else
                add (d[w],v,w) to PriorityQ
```

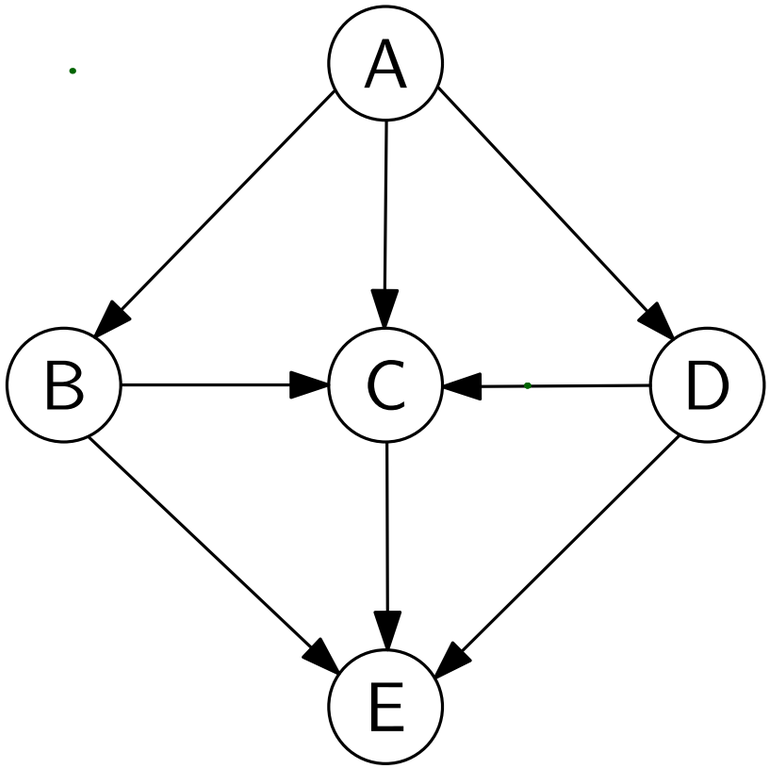
Priority-First Search Algorithm: Nonnegative Edges

```
Algorithm: Priority_First_Search (search_problem)
foreach v ∈ V do d[v] ← ∞
parent[s], d[s] ← -, 0
foreach v ∈ V do add (d[v],-,v) to PriorityQ
while PriortyQ is not empty do
    u, v ← remove edge (u,v) with minimum d[v] in PriorityQ
    foreach successor w of v do
        if d[v] + c(v,w) < d[w] then
            d[w] ← d[v] + c(v,w)
            parent[w] ← v
            update (d[w],v,w) in PriorityQ
return failure
```

Uniform Cost Search

```
Algorithm: Priority_First_Search (search_problem)
foreach v ∈ V do d[v] ← ∞
parent[s], d[s] ← -, 0
foreach v ∈ V do add (d[v],-,v) to PriorityQ
while PriortyQ is not empty do
    u, v ← remove edge (u,v) with minimum d[v] in PriorityQ
    if v is a goal node then return solution
    foreach successor w of v do
        if d[v] + c(v,w) < d[w] then
            d[w] ← d[v] + c(v,w)
            parent[w] ← v
            update (d[w],v,w) in PriorityQ
return failure
```

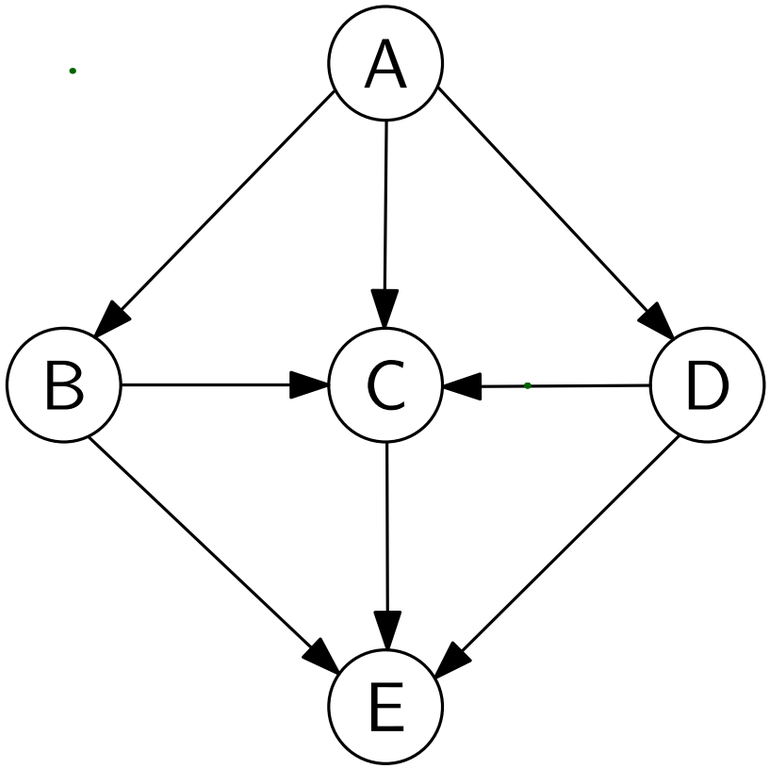
State Graph Search: First Expanded Path



I F

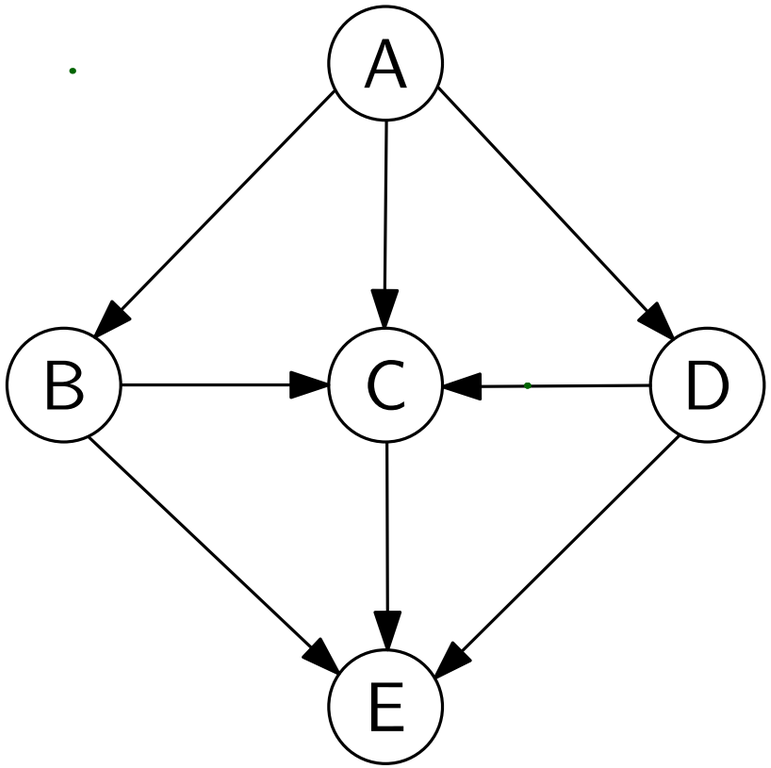
– A

State Graph Search: First Expanded Path



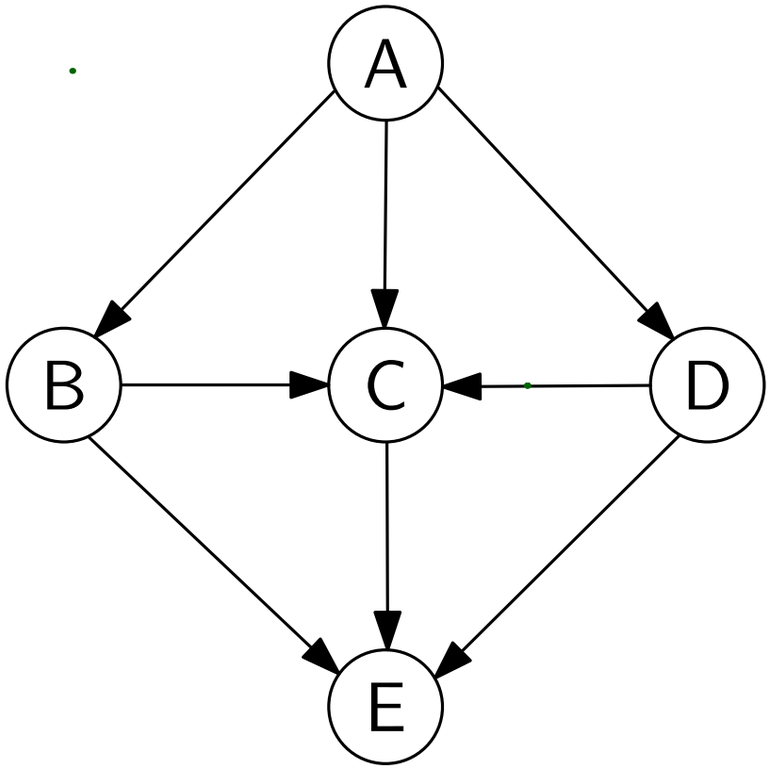
I	F
	– A
– A	AB, AC, AD

State Graph Search: First Expanded Path



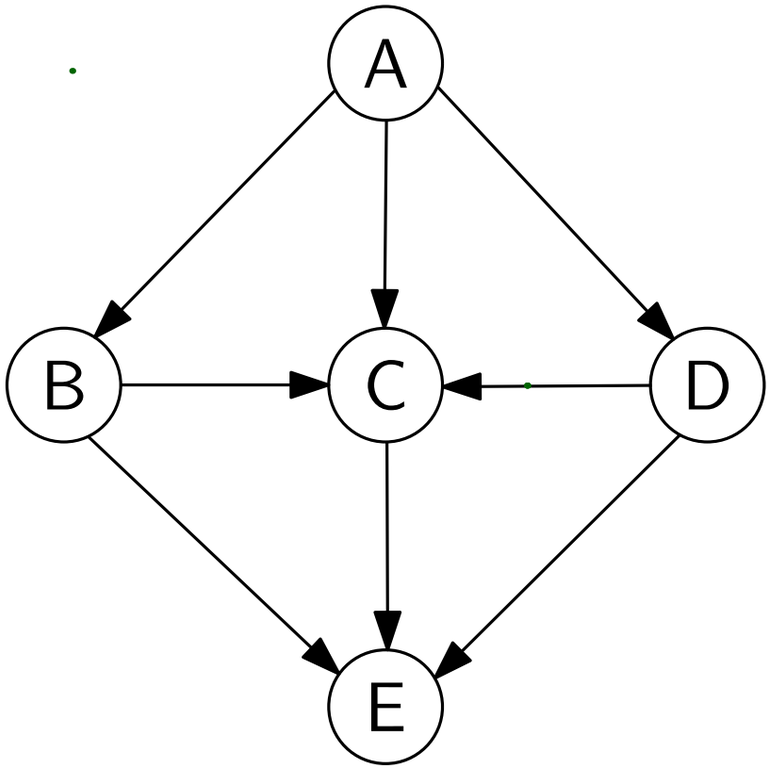
I	F
	-A
-A	AB, AC, AD
AC	AB, AD, CE

State Graph Search: First Expanded Path



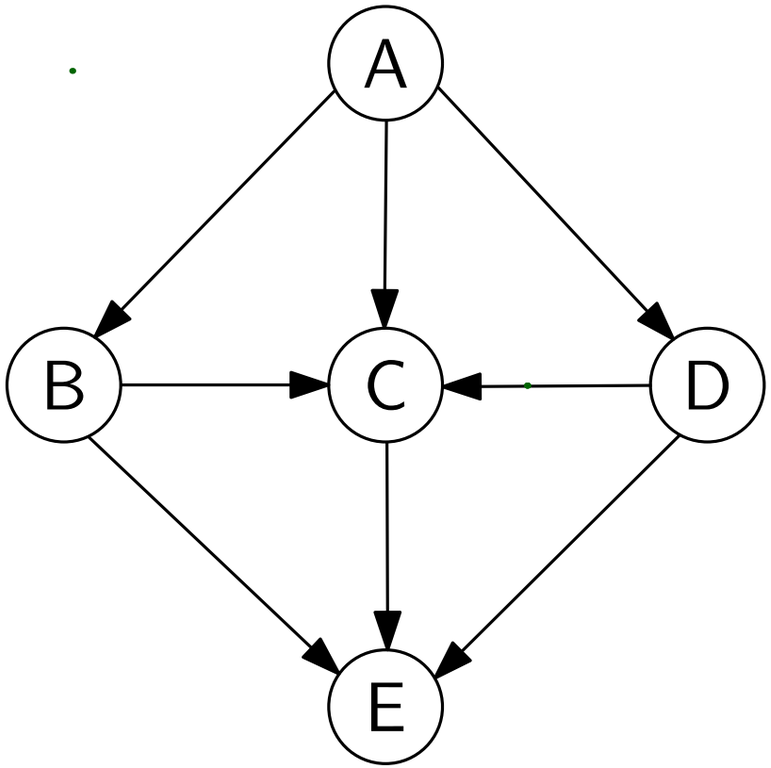
I	F
	A
A	AB, AC, AD
AC	AB, AD, CE
AB	AD, CE, BC, BE

State Graph Search: First Expanded Path

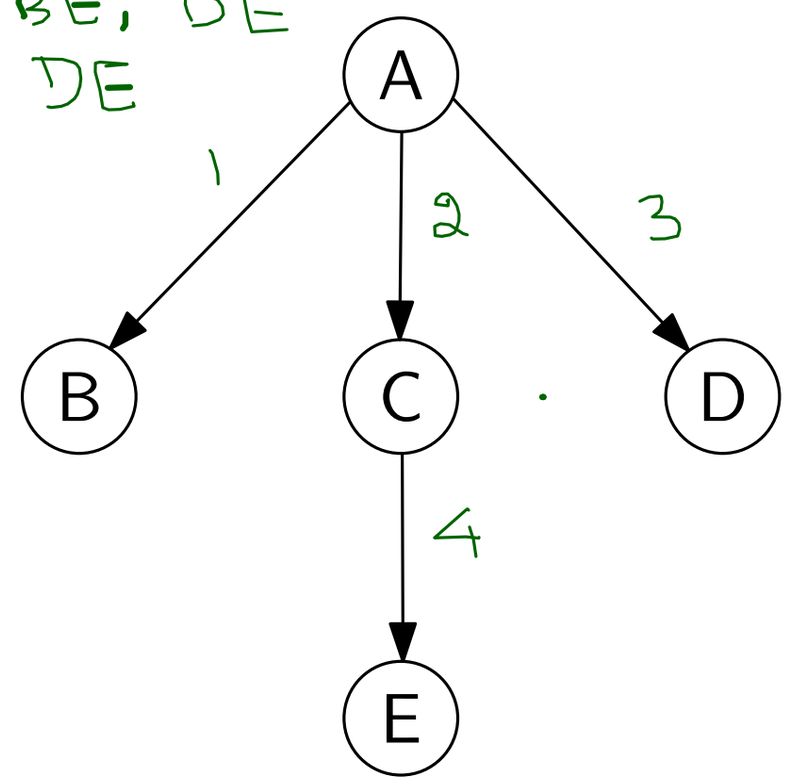


I	F
	-A
-A	AB, AC, AD
AC	AB, AD, CE
AB	AD, CE, BC, BE
AD	CE, BC, BE, DE

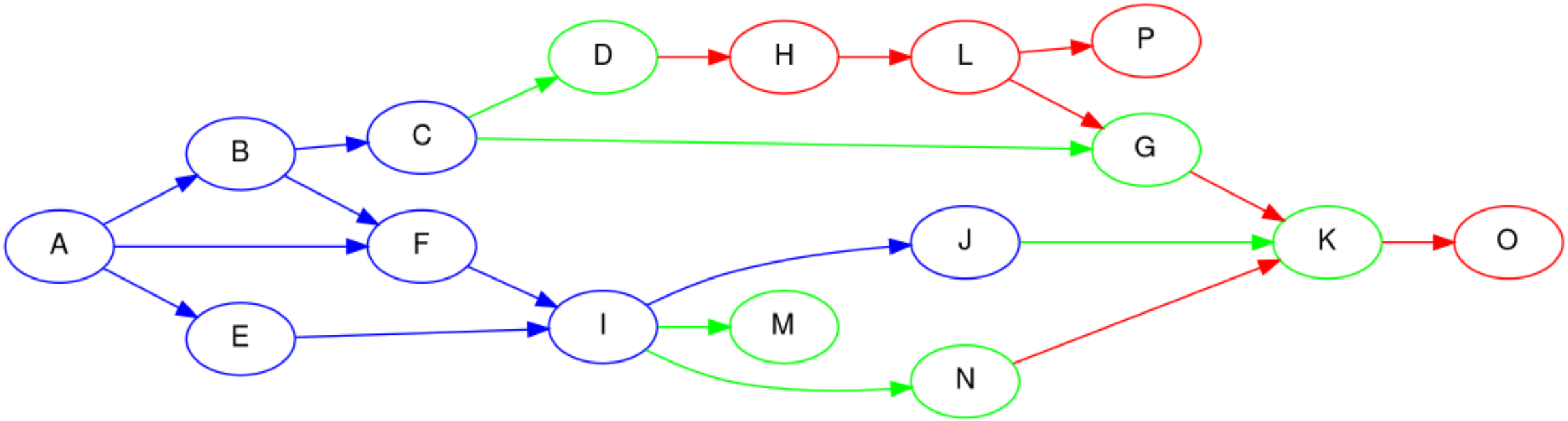
State Graph Search: First Expanded Path



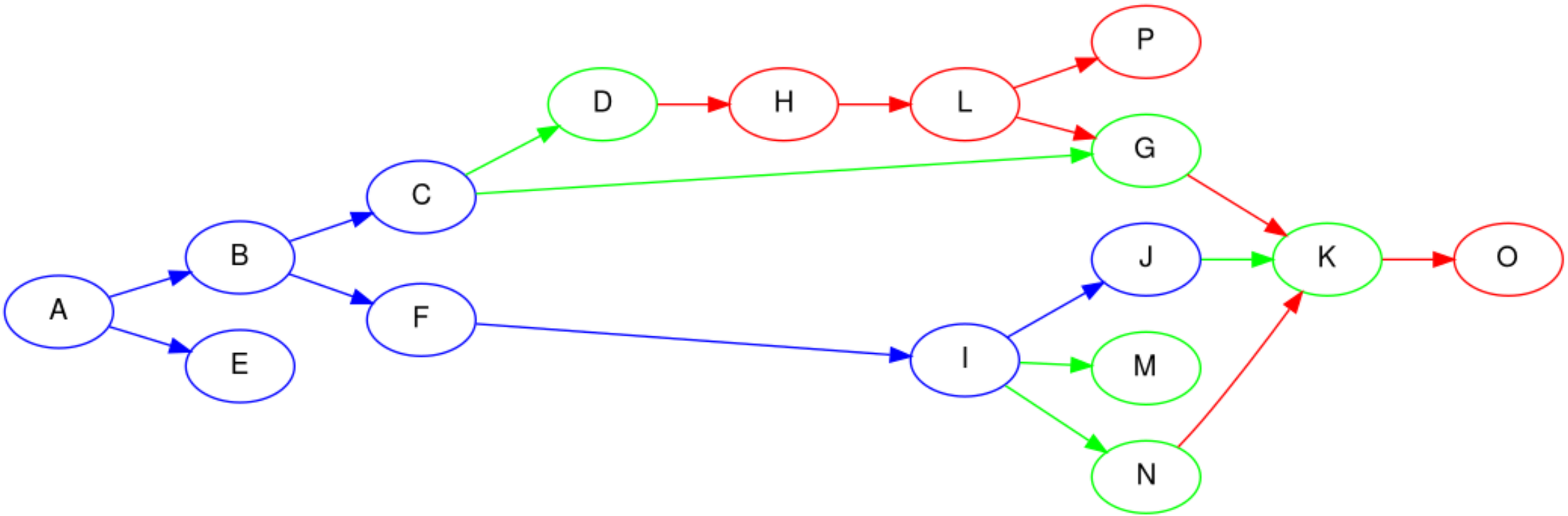
I	F
	A
A	AB, AC, AD
AC	AB, AD, CE
AB	AD, CE, BC, BE
AD	CE, BC, BE, DE
CE	BC, BE, DE
~~BC~~	BE, DE
~~BE~~	DE
~~DE~~	



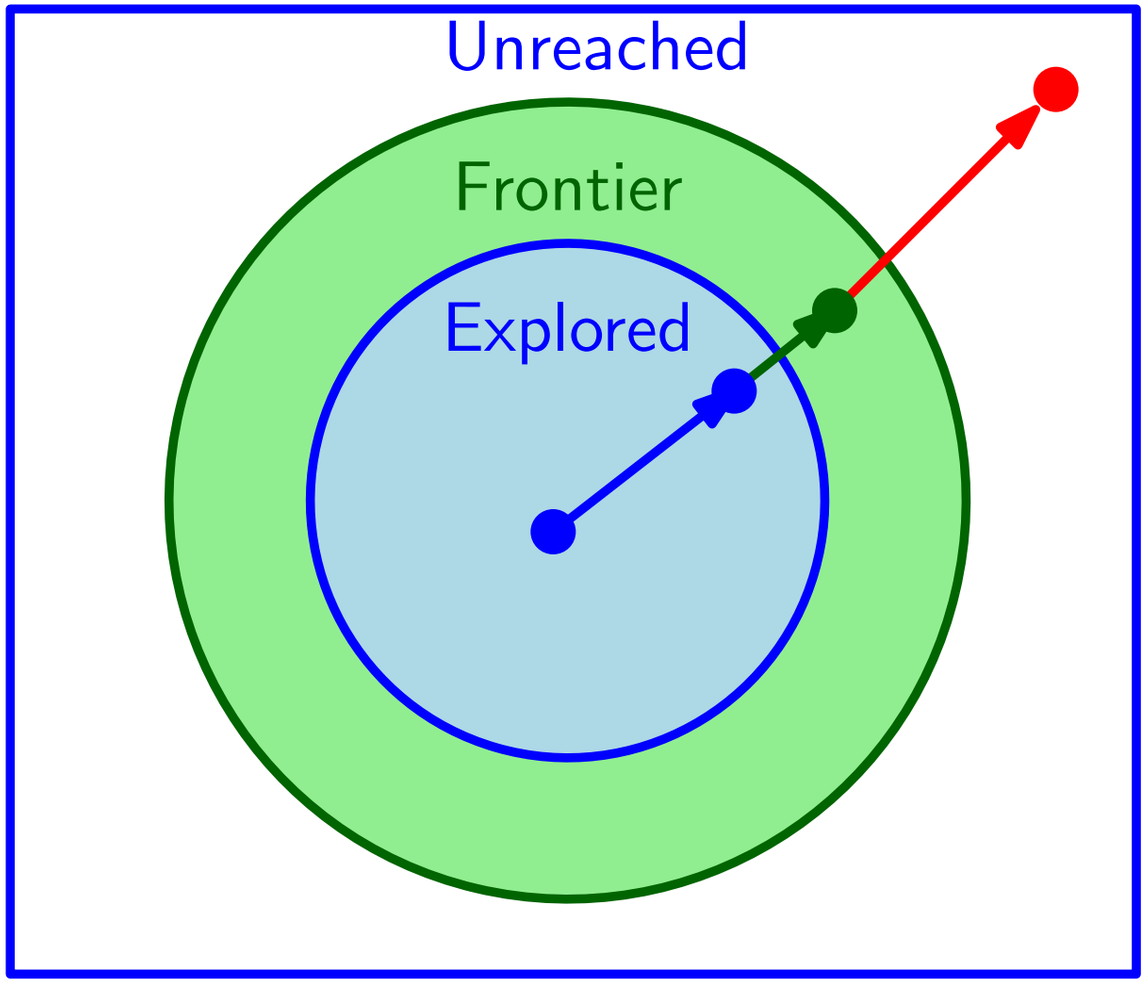
Explored, Frontier, Unreached



Explored, Frontier, Unreached



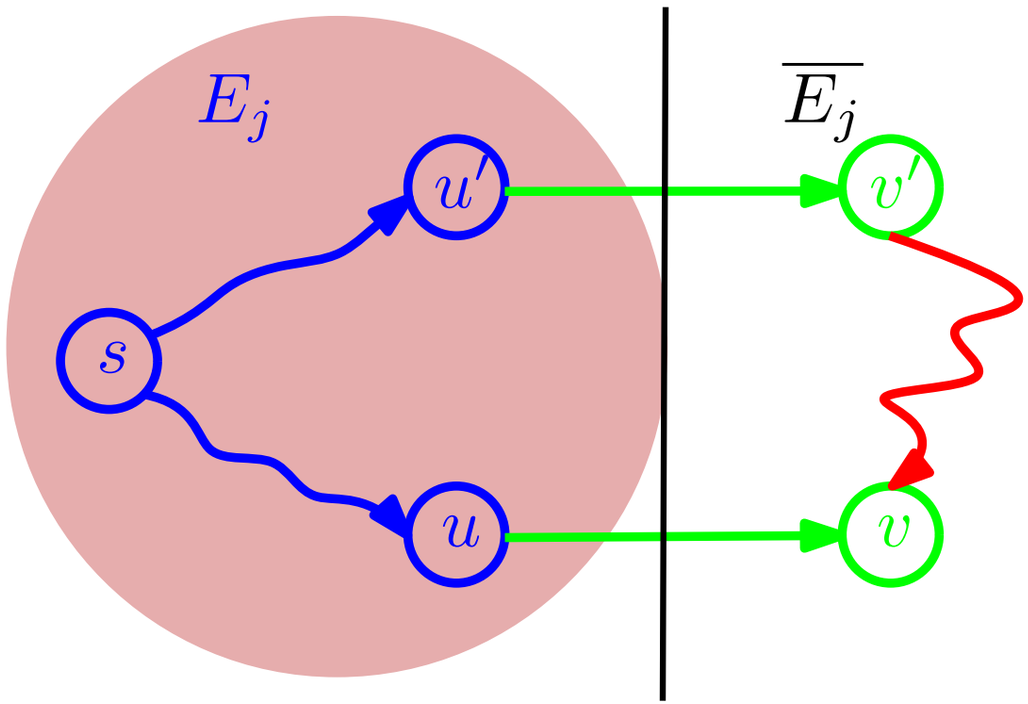
Explored, Frontier, Unreached



- Explored nodes have their paths in the path tree and their unexplored successors in the frontier.
- Frontier are the unexplored successors of the explored nodes. Edges/paths from the explored nodes to unexplored nodes.
- Unreached nodes neither in frontier nor in explored

- Reached = Frontier ∪ Explored
- When the algorithm explores a node, the path to the node is added to the path tree / committed.

UCS Correctness

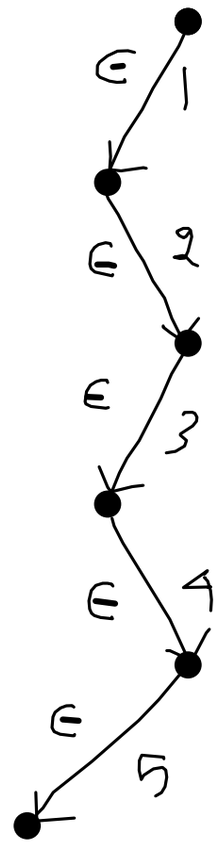


- Algorithm selects the shortest frontier path, say $s \rightarrow u \rightarrow v$
- An explored node's (blue) successor is always a frontier node (green).
- Every unexplored path $s \rightarrow u' \rightarrow v' \rightarrow v$ can be deconstructed into two subpaths
  - a frontier path (blue nodes followed by a green node) as a prefix $s \rightarrow u' \rightarrow v'$
  - a suffix of unexplored nodes $v' \rightarrow v$

- Every frontier path $s \rightarrow u' \rightarrow v'$ is longer than $s \rightarrow u \rightarrow v$
- Every unexplored path $s \rightarrow u' \rightarrow v' \rightarrow v$ is longer than $s \rightarrow u \rightarrow v$
- $s \rightarrow u \rightarrow v$ is the next shortest path.

UCS Performance

- Edge costs are nonnegative and bounded $\epsilon > 0$
- Let the depth of goal node k
- Cost of shortest path to goal $C = k\epsilon$
- $k = \frac{C}{\epsilon}$
- Maximum branching factor b
- Paths to be explored b^k
- Path length k
- Complexity $O(kb^k)$
- More work than BFS



$k = 5$

$C = k\epsilon$

$k = \frac{C}{\epsilon}$

UCS Properties

- UCS computes the shortest paths of the nodes.
- UCS computes the shortest paths in increasing order of their distance from the start node
- Loop invariant: Computed shortest distances $g(n)$ of the explored nodes are the actual shortest distances from the start node.
- Prove that the shortest frontier path is the next shortest path.

Problem: UCS orders nodes by distance from source to n
Goal: take into account distance from n to goal also.

End